FlashStates: Unlocking Flexible Hybrid State Management for Agentic LLM Inference

Chenxi Li[1,*], **Weili Xu**[2,*,†], **Tianyi Zhang**[1], **Jue Wang**[1], **Zhongzhu Zhou**[1], **Chenfeng Xu**[3]

[1]Together AI, [2]University of Illinois at Urbana-Champaign, [3]University of Texas at Austin
[*]Equal contribution, [†]Project lead

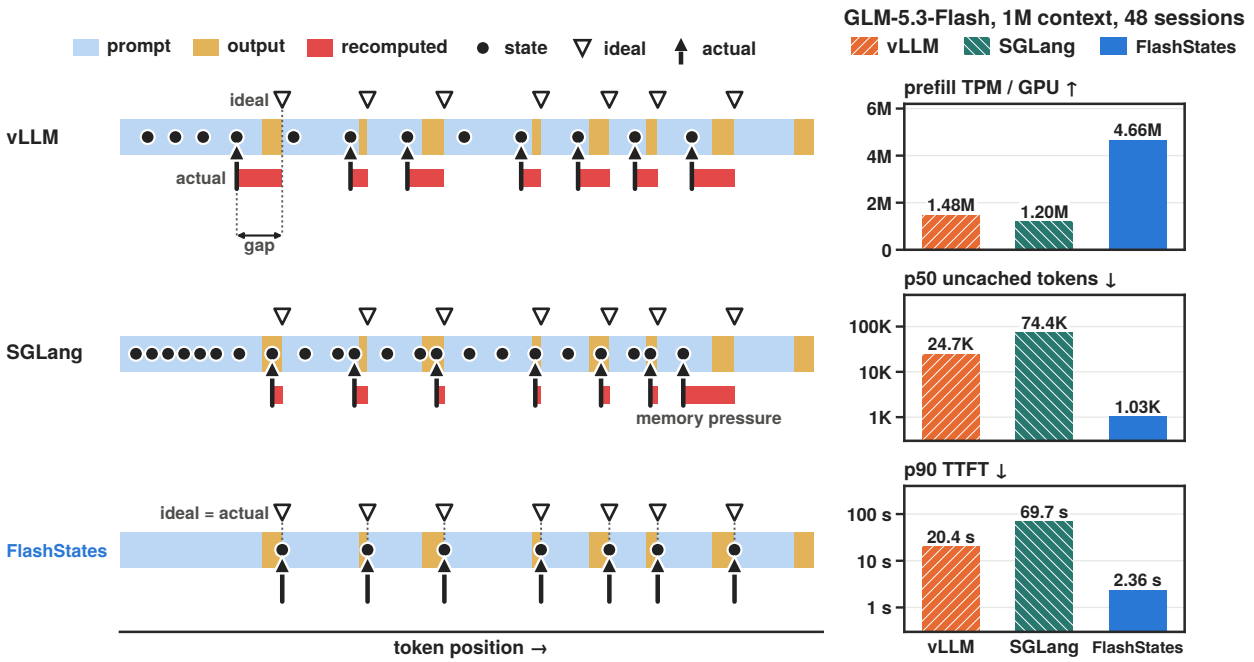
Efficient serving of agentic large language model (LLM) workloads relies on prefix caching to reuse long prefixes shared across requests. Hybrid models complicate this reuse, requiring both a recurrent checkpoint at the exact resume position and the matching attention key-value (KV) tensors. Existing serving systems offer limited flexibility in hybrid state management, resulting in either coarse-grained prefix reuse or increased contention for cache capacity. These limitations cause redundant prefill and reduce serving throughput. We present **FlashStates**, a serving system that enables flexible hybrid state management. FlashStates introduces Arbitrary State Checkpointing (ASC), a technique that decouples recurrent checkpoint positions from KV-cache block boundaries. Building on ASC, we formulate **Agentic Hybrid State Management** as a memory-constrained optimization problem to minimize redundant prefill across requests, and design a unified, cost-aware online policy that achieves fine-grained state reuse without dense checkpoint storage. When serving coding-agent workloads with GLM-5.3-Flash, Nemotron 3 Super and Qwen3.8-Flash-Next on NVIDIA B200 GPUs, FlashStates achieves up to 3.14×, 2.27× and 1.54× the prefill throughput of vLLM respectively.

Correspondence: Chenfeng Xu at cxu@utexas.edu
Code: https://github.com/FutureMLS-lab/FlashStates
Project & Demo: https://flashstates.ai

1 Introduction

The landscape of large language model (LLM) applications is rapidly shifting from traditional chat-oriented workloads toward agentic applications, including coding, enterprise assistance, and deep research (Aubakirova et al., 2026; Chang et al., 2026; Arya et al., 2026). This shift introduces two important workload characteristics. First, **growing contexts with substantial prefix reuse:** agentic workflows involve iterative model invocations interleaved with tool executions (Yao et al., 2023; Yang et al., 2024a), where successive requests extend previous contexts with generated outputs and tool results, often sharing long computed prefixes (Zhu et al., 2026). Second, **increasing prefill overhead:** As agent trajectories grow longer, recomputing shared prefixes becomes increasingly expensive, substantially increasing end-to-end task latency when cached computation cannot be reused. Together, these characteristics make efficient prefix caching and reuse critical for agentic LLM serving (Kang et al., 2026; Yuan et al., 2026).

Meanwhile, linear attention (Katharopoulos et al., 2020; Yang et al., 2024b) has emerged as a promising approach to improving LLM serving efficiency. To reduce decoding latency and alleviate KV cache memory overhead, recent open-weight LLMs increasingly adopt hybrid architectures that replace a subset of full-attention layers with linear-attention or state-space layers (Kimi Team et al., 2026; Z.ai, 2026; Qiu et al., 2026; Chandiramani et al., 2026; Dao and Gu, 2024). However, this architectural shift introduces a fundamental tradeoff: **while linear attention improves decoding efficiency, it complicates prefix reuse**. For full-attention models, computation can resume using the cached attention key-value (KV) tensors of a shared prefix. Hybrid models, however, additionally require a recurrent checkpoint at the exact resume position. Unlike attention KV tensors, which can be truncated to an earlier position, a recurrent checkpoint at a later position cannot be rolled back to recover one at an earlier position (Pan et al., 2025). Reusing a prefix therefore requires the attention KV of all its tokens and a recurrent checkpoint at its end, which summarizes the whole prefix. We call these two components the *hybrid state* of the prefix (Appendix A). Under high concurrency, where



(a) Checkpoint placement and agent context growth pattern

(b) Serving performance

Figure 1 Checkpoint placement and serving performance. (a) vLLM resumes from the last block-aligned checkpoint and recomputes up to the end of the previous turn; SGLang checkpoints more densely but runs under memory pressure; FlashStates resumes from the exact end of the previous turn. (b) Serving coding-agent workloads with GLM-5.3-Flash at 1M context and 48 concurrent sessions on NVIDIA B200 GPUs: FlashStates delivers 3.14× the prefill throughput of vLLM and 3.88× that of SGLang, and reduces p90 TTFT by 88.4% relative to vLLM.

numerous agent trajectories compete for limited memory, these dependencies further exacerbate memory pressure, redundant computation, and inefficient cache utilization.

Unfortunately, conventional LLM serving systems (Kwon et al., 2023; Zheng et al., 2024) were primarily designed for full-attention architectures, rather than the unique state dependencies of hybrid models. Hybrid state management requires three distinct decisions: **recurrent-checkpoint placement** determines where exact hybrid state can be materialized; **hybrid-state admission** determines whether a newly materialized hybrid state remains cached; and **hybrid-state eviction** determines which cached hybrid state leaves under memory pressure. Existing KV-centric designs couple hybrid state management to the granularity of KV-cache blocks, limiting its flexibility in two ways. First, recurrent-checkpoint placement is restricted to KV-cache block boundaries. Second, eviction reclaims individual KV-cache blocks without jointly accounting for all components required to reuse a prefix. This lack of flexibility introduces two major inefficiencies:

Coarse recurrent-checkpoint placement. When recurrent-checkpoint placement is restricted to KV-cache block boundaries, the engine can resume computation only from a limited set of positions. If the ideal resume position, corresponding to the longest reusable prefix, falls between these positions, the engine must fall back to an earlier position whose complete hybrid state remains cached, even when the attention KV tensors for the intervening tokens remain cached. As a result, previously processed tokens must be recomputed, introducing unnecessary prefill overhead and reducing serving efficiency (Figure 1a; Section 2.1).

Dependency-blind eviction. KV-centric eviction may retain the attention KV tensors of a prefix while evicting the recurrent checkpoint required to reuse it. This occurs when individual KV-cache blocks are reclaimed without jointly accounting for the dependencies among the components of a hybrid state. When a subsequent request matches the prefix, the missing recurrent checkpoint leaves its hybrid state incomplete and forces the engine to fall back to an earlier position whose complete hybrid state remains cached. Consequently, memory remains occupied by KV tensors that cannot be fully exploited for prefix reuse (Figure 2c; Section 2.2).

At first glance, these inefficiencies appear to admit a straightforward solution: materializing recurrent checkpoints at finer-grained positions to enable more precise prefix reuse. However, each retained hybrid state keeps its recurrent checkpoint and dependent attention KV available under a finite cache budget. As agent trajectories grow, retaining more hybrid states increases memory pressure and eviction frequency, potentially offsetting the benefits of finer-grained reuse (Sections 2.2 and 5.5). Therefore, the key challenge is not simply where recurrent checkpoints can be materialized, but which newly materialized hybrid states to admit and which cached hybrid states to evict to maximize computation savings under a constrained memory budget.

Recent systems, including FreeToken (Yang et al., 2026), Marconi (Pan et al., 2025), and Sparse Prefix Caching (Shirokikh and Nikolenko, 2026), have begun exploring recurrent-checkpoint placement and hybrid-state retention for future reuse. However, balancing computation savings against memory costs remains challenging and often requires carefully tuned heuristics. For example, Marconi assigns cached prefixes an eviction score that combines recency with FLOP efficiency, defined as the computation saved per byte of cached memory, using a tunable weighting parameter α. Its serving throughput varies by up to 16% across different choices of α, while its online tuner, which replays historical requests once during serving, does not consistently identify the optimal configuration (Appendix F.1). This sensitivity highlights a key limitation of heuristic-based cache management: the optimal balance between computation reuse and memory efficiency depends on workload characteristics, making fixed or manually tuned policies difficult to generalize.

To address these challenges, we introduce FlashStates, a serving system that enables flexible hybrid state management and exploits this flexibility through a unified, cost-aware policy. Rather than treating hybrid-state admission and eviction as separate heuristic decisions, FlashStates unifies them under a single priority (Equation (7)). This priority combines estimated prefill savings per unit of reclaimable cache capacity with online reuse history, guiding both decisions without workload-dependent parameter tuning. By jointly optimizing these decisions, FlashStates enables finer-grained prefix reuse while efficiently managing the limited GPU memory available under highly concurrent agentic workloads. This work makes three contributions:

A formulation of agentic hybrid state management. We formulate hybrid-state admission and eviction as a memory-constrained optimization problem (Section 4.1). The objective is to minimize the aggregate redundant prefill cost incurred between requests' actual and ideal resume positions (Equation (4)). This formulation connects recurrent-checkpoint placement, state–KV dependencies, and cache capacity through their effect on the computation that subsequent requests must repeat (Equation (5); Figures 2 and 3).

An online hybrid state management method. Under this formulation, we develop a solution that combines flexible recurrent-checkpoint placement with an online management algorithm. Arbitrary State Checkpointing (ASC) provides this flexibility by decoupling recurrent-checkpoint placement from KV-cache block boundaries, so that hybrid state can be materialized at any token position (Section 3). Building on ASC, the algorithm applies one dependency-aware priority to admit newly materialized hybrid states and to evict cached hybrid states (Section 4.3). It adapts GreedyDual-Size (GDS) (Cao and Irani, 1997) to combine online reuse with estimated avoided prefill per unit of capacity that eviction would actually free, while accounting for shared attention KV (Figure 4). The prefill-cost model is fitted once per model from measured prefill times and stays fixed across workloads (Appendix C).

A production integration across hybrid architectures. We integrate these mechanisms and the management algorithm into vLLM, a production serving engine. The implementation coordinates hybrid-state capture, prefix matching, memory allocation, and request lifecycle management to support safe state reuse across different hybrid architectures (Appendices G, H, and J). Supported models (Table 2; Section 5.3) include GLM-5.3-Flash, which combines DeepSeek Sparse Attention (DeepSeek-AI et al., 2025) with Kimi Delta Attention (KDA) (Kimi Team et al., 2025); Qwen3.8-Flash-Next, which combines Qwen Sparse Attention (QSA) with Gated DeltaNet (GDN) (Yang et al., 2025); and Nemotron 3 Super, which combines Grouped Query Attention (GQA) (Ainslie et al., 2023) with Mamba-2 (Dao and Gu, 2024).

We evaluate FlashStates on coding-agent workloads across GLM-5.3-Flash, Nemotron 3 Super, and Qwen3.8-Flash-Next on NVIDIA B200 GPUs (Section 5). With throughput measured in prefill tokens per minute (TPM), FlashStates achieves up to **3.14**×, **2.27**×, and **1.54**× the throughput of vLLM (Figures 1b and 5), while reducing p90 time to first token (TTFT) by **88.4%**, **77.1%**, and **79.0%**, respectively. When complemented with MoonCake (Qin et al., 2025), the FlashStates serving stack improves throughput by up to 41.4% and reduces p90 TTFT latency by up to 81.6% (Appendix D).

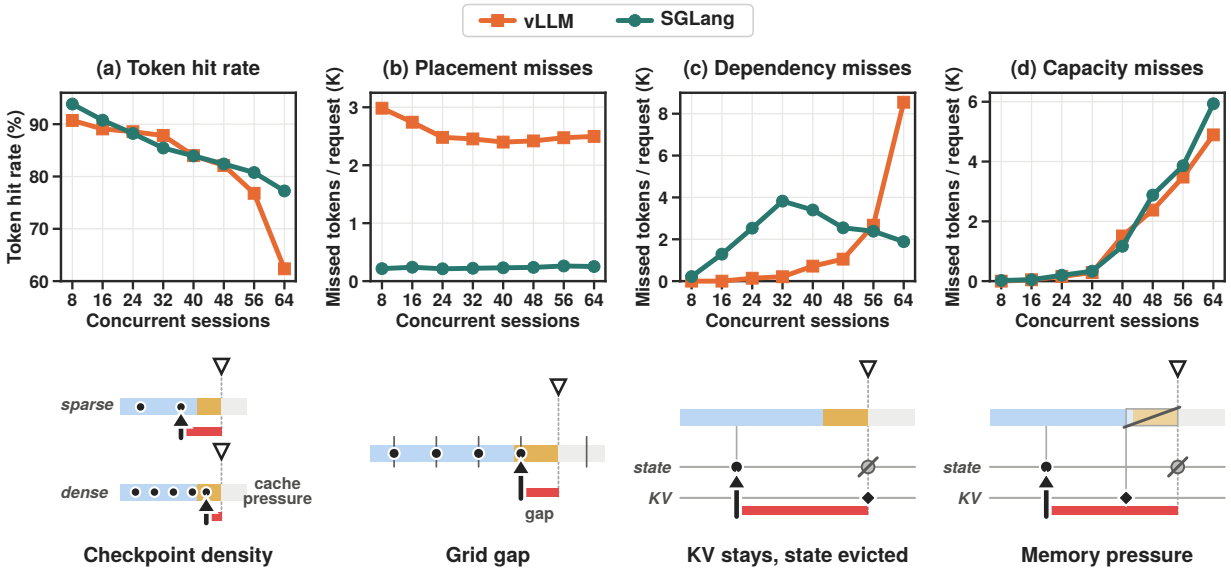


Figure 2 Cache hit rate and cache-miss attribution of vLLM and SGLang serving coding-agent workloads at increasing concurrency. Both engines serve GLM-5.3-Flash with the same cache memory at 8–64 concurrent sessions. The top row shows (a) the token hit rate and (b–d) the redundant prefill tokens per request from three kinds of misses, each sketched below its panel. Each sketch follows one request that has already processed its prompt (blue) and output (yellow). The request should resume where they end, but it resumes at an earlier recurrent checkpoint and recomputes the span in between (red). (b) Placement misses arise because both engines save recurrent checkpoints only at KV-cache block boundaries, so the request recomputes the gap after the last saved checkpoint. As the sketch below (a) shows, saving checkpoints more densely shortens this gap at the cost of cache memory. (c) Dependency misses arise when the checkpoint where the request should resume is evicted although its KV is still cached. (d) Capacity misses arise when memory pressure evicts that checkpoint together with the KV of the latest tokens. Below 40 sessions, placement and dependency misses dominate, and capacity misses grow only from 40 sessions on.

2 Motivation

In this section, we profile vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024) on agentic coding workloads to quantify the two problems in Section 1, and show that neither a coarse nor a fine recurrent-checkpoint grid avoids them. We attribute every computed prompt token to one root cause by comparing each request's cached prefix with its ideal resume position (Equation (2) and Appendix B). *New content* lies beyond the ideal resume position and must be computed under any caching mechanism. A *placement miss* lies between the ideal resume position and the nearest recurrent checkpoint the engine ever saved before it on the prefix. A *dependency miss* is recomputed because a saved recurrent checkpoint was evicted while the attention KV it depends on stayed cached.[1] A *capacity miss* is recomputed because both a recurrent checkpoint and part of its KV were evicted under cache capacity contention. Figure 2 reports the hit rate and the three avoidable causes per request.

2.1 Coarse Recurrent-Checkpoint Placement

When serving GLM-5.3-Flash, vLLM saves a recurrent checkpoint only at 4,352-token KV-cache block boundaries inside the prompt (Zhang et al., 2025; vLLM Project, 2026; vLLM, 2026c), and never during decoding or at the end of a request; two open pull requests propose retaining decode-end checkpoints (vLLM, 2026a,b). A follow-up request extends the full prompt and output of its predecessor, yet it can resume only from the last KV-cache block boundary of the previous prompt. At low concurrency (8–16 sessions), vLLM evicts almost nothing (Figure 2d), so placement misses are its only avoidable misses, accounting for up to 43% of vLLM's uncached prompt tokens (Figure 2b).

[1] We observe no misses in the reverse direction, where a recurrent checkpoint survives but its KV does not.

A naive solution is to use a finer grid, which SGLang adopts. SGLang aligns recurrent checkpoints to a 256-token grid of KV-cache blocks (SGLang, 2026b). Its placement misses drop to about 0.2K tokens per request, and its cache hit rate exceeds vLLM's by up to 3.1% at low concurrency (Figure 2a). Coarse placement therefore explains the gap. Finer placement, however, is not free, because every additional recurrent checkpoint increases contention for cache capacity.

2.2 Dependency-Blind Hybrid-State Eviction

vLLM and SGLang both evict recurrent checkpoints by recency alone, without regard to the attention KV they depend on (Kang, 2026). SGLang keeps about 430 recurrent checkpoints in a separate pool beside 5.9M tokens of KV and evicts each pool in its own least-recently-used (LRU) order (SGLang Team, 2026a,b). At 32 sessions, SGLang's finer grid saves 4.2× as many checkpoints per request as vLLM, so the state pool fills and evicts checkpoints whose KV is still cached (SGLang, 2026a). These dependency misses cause 33–43% of SGLang's uncached prompt tokens at 24–32 sessions (Figure 2c). Affected requests recompute a median of 66K–68K tokens, nearly the whole prefix, although the prefix's KV stays in GPU memory. Capacity misses stay below 0.4K tokens per request at these loads (Figure 2d), so the loss stems from ignored dependencies rather than limited capacity. SGLang's hit rate therefore drops below vLLM's at 24 and 32 sessions (Figure 2a).

vLLM keeps fewer checkpoints but follows the same rule. In vLLM, a request's prompt-end checkpoint returns to the evictable queue at the request's first decoding step, while the request is still running, and is later evicted by recency. vLLM's dependency misses grow from 40 sessions and reach 8.5K tokens per request at 64 sessions, where its hit rate drops to 62.4% (Figures 2a and 2c). From 40 sessions on, vLLM and SGLang both enter hybrid state thrashing, where capacity misses add to dependency misses (Figure 2d).

Both engines therefore face the same limitation: placement remains tied to KV-cache block boundaries, finer placement creates more hybrid states for the same budget, and recency-based block eviction ignores the dependencies that make those states usable. Thus, a finer grid only trades placement misses for dependency misses as concurrency grows. Efficient serving requires three decoupled decisions: recurrent-checkpoint placement determines where hybrid state can be materialized, hybrid-state admission decides what remains cached, and hybrid-state eviction decides what leaves under pressure. Section 3 decouples recurrent-checkpoint placement from KV-cache block boundaries, so a recurrent checkpoint can be saved at any token position. Section 4 then chooses which hybrid states to retain, using one dependency-aware GDS priority for both admission and eviction.

3 Arbitrary State Checkpointing

To address the placement misses that force later agent turns to recompute prompt tails and generated outputs (Section 2.1), we introduce ASC. ASC expands the set of positions where hybrid state can be materialized by decoupling recurrent-checkpoint placement from two existing boundaries. A KV-cache block boundary marks the end of a fixed-size block of attention KV; a prefill-call boundary marks the end of a prefill invocation. A reusable prefix need not end at either boundary. Section 3.1 captures the hybrid state of a request's processed prefix at completion, without changing the KV-cache block size. Section 3.2 exports the required recurrent checkpoint from the kernel backend without splitting the prefill call.

3.1 Exact Hybrid-State Capture

Block-aligned caching ties the granularity of prefix reuse to the KV-cache block size (Zhang et al., 2025). To overcome this constraint and let later requests reuse a processed prefix at its exact endpoint, FlashStates captures its hybrid state when the request completes. ASC submits the state components held by the request together for hybrid-state admission (Section 4). If admitted, they remain in their existing KV-cache blocks; capture itself copies no model data.

The captured recurrent checkpoint corresponds to the exact endpoint of the request's processed prefix, and all other state components match that same prefix. Even if the endpoint lies inside a KV-cache block, a later request that hits this hybrid state can resume there without falling back to a block boundary and recomputing the prefix tail. To support this reuse, FlashStates retains the whole terminal KV-cache block but treats only

Table 1 Summary of notation for hybrid state management. Each cached hybrid state is characterized by its prefix length, the cache allocations it requires, its restore cost, and the capacity its eviction releases.

Notation	Description
X_r, n_r, τ_r	Prompt of request r, its length, and its lookup time
$\ell_r^\star$, ℓ_r^π	Ideal and actual resume positions of request r under policy π
$C(a, b)$	Estimated full-model prefill time from prefix position a to b
Π_{ASC}	Causal policies that retain hybrid state only where ASC can materialize it
B, $m(o)$	Cache budget; byte size of cache allocation o
$\mathcal{M}_\pi(\tau)$	Cache allocations resident at time τ under policy π
u, $p(u)$, $D(u)$	Cached hybrid state, its prefix length, and the cache allocations it requires
$f(u)$, $g(u)$	Fallback position of u; restore cost $C(f(u), p(u))$
$F(u)$, $s(u)$	Allocations that evicting u releases; their number, the released capacity
$\rho(u)$	Cost density $g(u)/s(u)$
Λ, stamp(u), $H(u)$	GDS watermark; watermark at the last use of u; priority of u

the tokens belonging to the captured prefix as valid. This decouples the exact reusable prefix position from physical KV-cache block boundaries while preserving the engine's original block size. Capturing a position inside a prefill call requires the export mechanism in Section 3.2.

3.2 In-Chunk State Export

To obtain a recurrent checkpoint inside a prefill call without interrupting the original call, FlashStates passes the target offset c to the backend and asks the kernel to export the checkpoint at that position as a side output while completing the original computation. To reduce global-memory traffic, existing no-export fused kernels typically maintain only the advancing live recurrent state and write S_{final} at the end of the call; even though the interior state S_c exists mathematically, it is not retained in a form readable by the serving system. This is a limitation of the kernel dataflow and interface, rather than of the linear attention recurrence itself. For the recurrence

$$S_t = F(S_{t-1}, x_t),$$

S_c is well defined at every prefix endpoint. Thus, as long as the kernel can directly obtain S_c at a position satisfying the kernel's tile and alignment requirements or construct it using computations already available in the current tile, the same export approach can be applied to different linear attention recurrences.

We implement this contract in both the KDA (Kimi Team et al., 2025) and GDN (Yang et al., 2025) kernels:

$$\text{PrefillExport}(X, S_0, c) \to (O, S_{\text{final}}, R_c), \tag{1}$$

Here, R_c equals the canonical recurrent checkpoint produced by the same backend when the call is truncated at c; O and S_{final} remain identical to those of the no-export call. KDA's export grid aligns with its compute tiles, so the kernel can directly serialize the live recurrent state already available at a tile boundary. GDN uses larger compute tiles, and states at interior positions are not directly materialized; the kernel uses operands already computed for that tile to perform one bounded prefix update, obtains R_c, and then continues the original full-tile computation. The two implementations differ internally, but each exports only one recurrent checkpoint, and the side output does not feed back into the live recurrent state. The tile layouts, alignment requirements, overheads, and validation scope are given in Appendix H.

After export, FlashStates combines R_c with attention KV for the same prefix, as defined in Section 3.1, to form an exact hybrid state at position $p = q + c$. Recurrent-checkpoint placement therefore no longer requires the target position to coincide with a prefill-call boundary: ASC makes the complete hybrid state at that position available for materialization, and hybrid-state admission then decides whether to retain it in the cache.

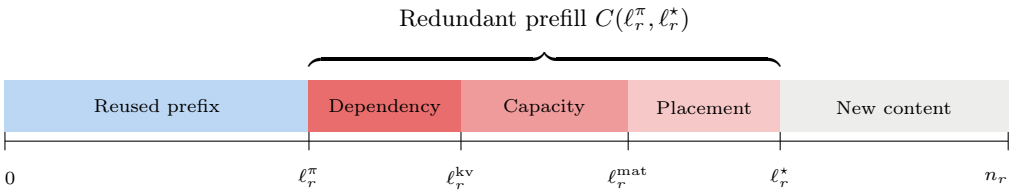


Figure 3 Redundant prefill of request r. Tokens before ℓ_r^π are reused, and tokens after $\ell_r^\star$ are new content. The tokens in between are recomputed, and Equation (5) attributes them to dependency, capacity, and placement.

4 Agentic Hybrid State Management

Building on ASC, this section chooses which hybrid states to retain under a finite cache budget. We formulate this choice as minimizing redundant prefill (Section 4.1), derive structural properties under simplifying assumptions (Section 4.2), and use these properties to design dependency-aware GreedyDual-Size (GDS) admission and eviction with one priority (Section 4.3). Table 1 summarizes the notation.

4.1 Problem Formulation

When a request resumes before its ideal position, it repeats *full-model prefill* over the intervening tokens: every model layer processes them again. Under a cache budget B, we choose an online admission-and-eviction policy π to minimize this cost across requests.

Resume positions. The *ideal resume position* is the end of the longest already processed prefix of a request's prompt. Request r has prompt X_r of length n_r and lookup time τ_r, when prefix matching runs. Let $\mathcal{P}_r$ contain every prefix processed before τ_r under the same model configuration, whether or not its hybrid state was retained. With LCP denoting the longest exact common-prefix length and ϵ the empty sequence, this position is

$$\ell_r^\star = \max_{Z \in \mathcal{P}_r \cup \{\epsilon\}} \mathrm{LCP}(X_r, Z), \tag{2}$$

which eviction cannot change.

A *cache allocation* is one KV-cache block (Appendix A). Cached hybrid state u represents an exact prefix of length $p(u)$; $D(u)$ is the set of cache allocations holding its components. Let $\mathcal{M}_\pi(\tau)$ be the distinct allocations resident at time τ under policy π. State u is *intact* at time τ when all its required allocations remain resident, $D(u) \subseteq \mathcal{M}_\pi(\tau)$. It is *usable* for request r when it is intact at τ_r and represents exactly the first $p(u)$ tokens of X_r. With $\mathcal{U}_r^\pi$ denoting these usable hybrid states, request r can resume from any $p(u)$ with $u \in \mathcal{U}_r^\pi$. The *actual resume position* is the one nearest to the ideal resume position, or 0 if none exists:

$$\ell_r^\pi = \max\big(\{0\} \cup \{p(u) : u \in \mathcal{U}_r^\pi\}\big). \tag{3}$$

Objective. Since $0 \le \ell_r^\pi \le \ell_r^\star \le n_r$, the request repeats full-model prefill over $[\ell_r^\pi, \ell_r^\star)$. Let $C(a, b) \ge 0$ estimate the full-model prefill time from prefix position a to b. The estimate is additive across adjacent intervals: $C(a, b) = C(a, a') + C(a', b)$ for $a \le a' \le b$, as in the cost model of Appendix C. Let $\mathcal{R}$ be the requests of a trace and $m(o)$ the byte size of allocation o. The design objective is

$$\underset{\pi \in \Pi_{\mathrm{ASC}}}{\text{minimize}} \quad \sum_{r \in \mathcal{R}} C(\ell_r^\pi, \ell_r^\star) \qquad \text{subject to} \quad \sum_{o \in \mathcal{M}_\pi(\tau)} m(o) \le B, \quad \forall \tau. \tag{4}$$

Policies in Π_{ASC} are *causal*: they use only past and current events. They retain hybrid state only where ASC can materialize it and preserve allocations required by *live requests*, meaning queued or running requests. A live request *holds* a cached hybrid state when it will resume from that state or still requires its allocations.

The budget B applies to memory holding cache allocations, excluding model weights and execution workspace; this memory can include a host-memory tier (Appendix D). Appendix I.1 gives the capacity accounting for shared blocks, private copies, and memory tiers.

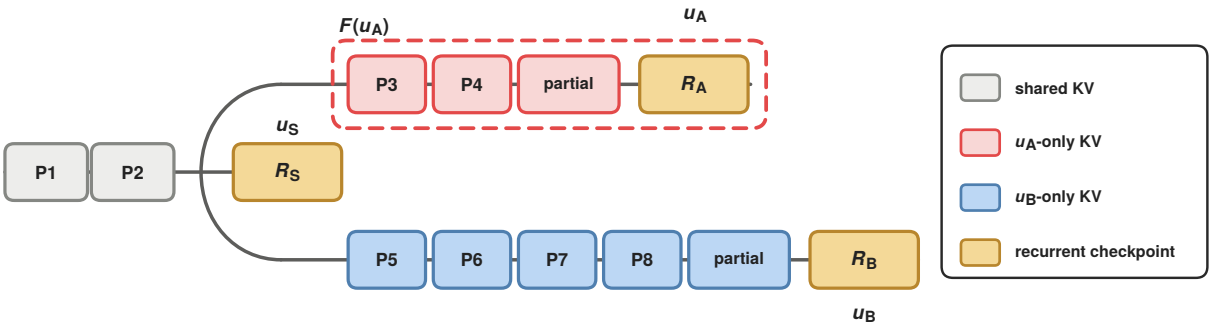


Figure 4 Allocations released by evicting hybrid state u_A. An earlier hybrid state u_S requires shared KV-cache blocks P1/P2 and recurrent checkpoint R_S; hybrid states u_A and u_B extend that prefix, and R_A is the recurrent checkpoint of u_A. Evicting u_A releases only R_A and the KV-cache blocks that only u_A requires, namely P3, P4, and its terminal KV-cache block. Counting the full prefix would overstate the released capacity of u_A and understate its cost density.

Three sources of redundant prefill. Consider prefixes of X_r that end at or before $\ell_r^\star$ and whose hybrid state was materialized before lookup time τ_r. Among them, the endpoint nearest to the ideal resume position is ℓ_r^{mat}. Among those whose attention KV remains resident at τ_r, the nearest endpoint is ℓ_r^{kv}. Each position is 0 if no prefix meets its corresponding conditions. A usable hybrid state has been materialized and retains its attention KV, so $\ell_r^\pi \le \ell_r^{\text{kv}} \le \ell_r^{\text{mat}} \le \ell_r^\star$. Additivity of C then splits redundant prefill into

$$C(\ell_r^\pi, \ell_r^\star) = \underbrace{C(\ell_r^{\text{mat}}, \ell_r^\star)}_{\text{placement}} + \underbrace{C(\ell_r^{\text{kv}}, \ell_r^{\text{mat}})}_{\text{capacity}} + \underbrace{C(\ell_r^\pi, \ell_r^{\text{kv}})}_{\text{dependency}}. \tag{5}$$

Figure 3 illustrates the three terms, which correspond to the placement, capacity, and dependency misses measured in Section 2 (Figures 2b–2d). ASC addresses the placement term by decoupling recurrent-checkpoint placement from KV-cache block boundaries. The capacity term arises when the budget forces out attention KV; the dependency term arises when that KV remains but another required component, such as its recurrent checkpoint, does not. Admission and eviction therefore control the latter two terms.

4.2 Structure of an Optimal Policy

The cost in Equation (4) depends on future requests. We use properties of an optimal policy under simplifying assumptions to guide the online design (Appendix I.1).

Restore cost and released capacity. When hybrid state u is evicted, blocks still required by another intact cached hybrid state or a live request remain allocated. Eviction frees only the remaining blocks in $D(u)$; denote this set by $F(u)$. Since KV-cache blocks have equal size (Appendix A), their number is the *released capacity* $s(u) = |F(u)|$ (Figure 4). For admission, we evaluate a newly materialized state as if it were cached.

After u is evicted, a request that would resume there falls back from $p(u)$ to the nearest earlier position with usable hybrid state, or 0 if none remains. We call this the *fallback position* $f(u)$. Repeating full-model prefill from $f(u)$ to $p(u)$ incurs the *restore cost*:

$$g(u) = C(f(u), p(u)). \tag{6}$$

With other decisions fixed, evicting u adds exactly $g(u)$ of redundant prefill for a request that would resume there (Lemma 1). For $s(u) > 0$, the *cost density* $\rho(u) = g(u)/s(u)$ is this extra prefill per released KV-cache block. It does not predict whether the state will be reused.

Properties of an optimal policy. For the capacity term, we use the *timer model* of time-to-live (TTL) caches (Ferragut et al., 2016; Dehghan et al., 2019). Each agent's conversation keeps at most its latest hybrid state while waiting for a tool, with a *timer* set at request completion. The conversation *returns* at the lookup of its next request; the elapsed time since completion is its *tool-call duration*. A return within the timer reuses the state; otherwise, it is evicted at expiry. A zero timer evicts immediately.

The model assumes that tool-call durations are independent draws from one distribution. It ignores rounding attention KV to whole KV-cache blocks and constrains average cache occupancy over time (Appendices I.1

and I.9). We call a resident allocation *dead* when no live request or intact cached hybrid state requires it. Appendix I.6 states the full timer model, and Appendix I proves three properties, one for each term of Equation (5).

1. Materializing hybrid state at every request end removes the placement term, because every later request ideally resumes where the previous request of its conversation ended (Proposition 1).
2. Some optimal policy evicts a cached hybrid state u by releasing exactly $F(u)$ and holds no dead allocation, so it has no dependency term (Proposition 2).
3. In the timer model, some optimal policy sets the timer of each hybrid state from its cost density alone, and a higher cost density never gets a shorter timer (Proposition 3).

Intuitively, a cached hybrid state is worth its KV-cache blocks while its conversation is still likely enough to return soon, and a higher cost density justifies a longer wait.

Mechanisms of FlashStates. ASC captures hybrid state at request completion (Section 3.1), and dependency-aware eviction releases exactly $F(u)$, implementing the first two properties. For the third, GDS maintains a *watermark*, the largest priority evicted so far (Section 4.3). In the timer model, a state without reuse remains cached until the watermark has risen by its cost density since its last use. Approximating this rise as steady in a large cache gives timers proportional to cost density without knowing the tool-call duration distribution (Appendix I.8).

LRU over whole hybrid states gives nearly the same timer to every state under its large-cache approximation (Che et al., 2002; Fricker et al., 2012). We compare the policies with full budgets and equal average occupancy under these two approximations. For common distributions of tool-call durations, GDS then saves at least as much prefill as this LRU in the timer model (Proposition 4; Appendix I.7 lists these distributions).

4.3 Dependency-Aware GDS Admission and Eviction

FlashStates uses one dependency-aware GDS priority for both admission and eviction.

Priority. We adapt GDS (Cao and Irani, 1997; Young, 2002) to hybrid states, using released capacity as size. We denote the watermark by Λ and initialize it to 0. When hybrid state u is first cached, a request resumes from it, or the last live request holding it ends, FlashStates records the current watermark as $\text{stamp}(u)$. We call each of these events a *use* of u. For $s(u) > 0$, each use sets its priority to

$$H(u) = \text{stamp}(u) + \rho(u). \tag{7}$$

The stamp is the watermark recorded at u's last use, while $\rho(u)$ is its restore cost per released block. Both terms have units of prefill time per KV-cache block, so they can be added without a workload-specific coefficient. Under cache pressure, successive evictions advance the watermark. A faster rise shortens how long a state without further reuse stays cached in the timer model (Appendix I.8), GDS thus adapts retention to demand.

Admission and eviction. The following rules govern GPU memory and treat each hybrid state as a whole. Near a full budget, caching a newly materialized state may displace one that saves more prefill. The new state takes the current watermark as its stamp and enters the same priority order. Without a free KV-cache block, it is not admitted if GDS would evict it next. When a request needs blocks and none is free, FlashStates evicts the *eligible* hybrid state u of minimum priority. Eligible states are cached, have positive released capacity, and are held by no live request. Eviction releases exactly $F(u)$ and raises Λ to $H(u)$ if $H(u)$ is higher. Appendix J details the admission test, the handling of dead allocations and hybrid states that are no longer intact, and priority updates.

5 Experiments

In this section, we evaluate FlashStates on coding and math agents with three hybrid models on NVIDIA B200 GPUs. Section 5.1 describes the setup. Section 5.2 presents the serving results and analyzes them through contention for the cache. Section 5.3 examines these gains across the three hybrid architectures, and

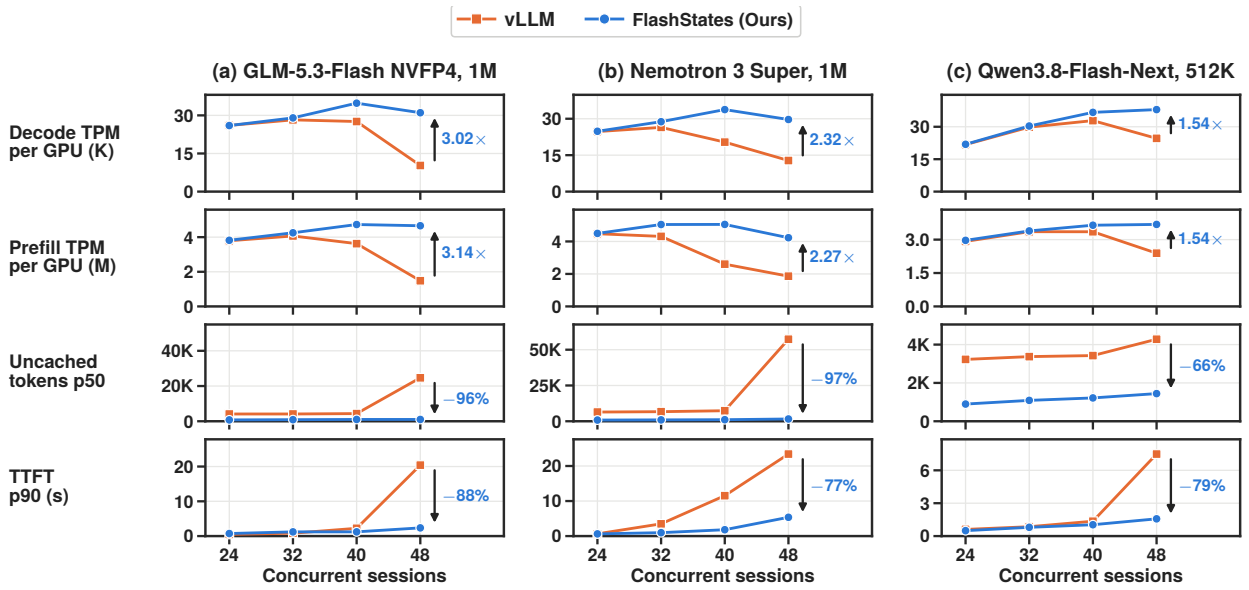


Figure 5 FlashStates sustains serving performance as concurrency grows on three hybrid architectures. vLLM and FlashStates serve coding-agent traces with the same cache budget. As concurrency grows, vLLM's throughput drops while FlashStates stays near its peak. At 48 sessions, FlashStates achieves up to 3.02× vLLM's decode throughput and 3.14× its prefill throughput, and reduces p90 TTFT by up to 88.4%.

Table 2 Models and workloads. Each engine runs on two B200 GPUs with tensor parallelism. Block is vLLM's KV block size in tokens, and vLLM saves recurrent checkpoints only at block boundaries. Cache is the per-GPU memory for hybrid state, which vLLM and FlashStates receive equally. Nemotron's cache budget provides approximately 10.45M tokens of capacity as reported by vLLM; Appendix C also evaluates a larger 60 GiB cache per GPU. Context limits use K = 2^{10} and M = 2^{20} tokens. For Qwen3.8-Flash-Next, trajectories longer than 512K tokens are excluded.

Model	Recurrent	Workload	Context	Block	Cache GiB/GPU	Traces
GLM-5.3-Flash (NVFP4)	KDA	Coding	1M	4,352	62.45	393
GLM-5.3-Flash (NVFP4)	KDA	Math	128K	4,352	62.45	100
Qwen3.8-Flash-Next (FP8)	GDN	Coding	512K	3,136	67.26	323
Nemotron 3 Super (FP8)	Mamba2	Coding	1M	8,320	22.46	393

Section 5.4 evaluates a decode-heavy math agent workload. Section 5.5 separates the performance gain from ASC and from hybrid state management.

5.1 Experimental Setup

Workloads. We evaluate two agent workloads. The coding workload replays the Claude Code trajectories released by SemiAnalysis (SemiAnalysis, 2026), which interleave sub-agents with pauses between requests. It is a **long-context** workload. With a 1M-token context limit, the mean prompt reaches 219K tokens (median 142K). The math workload runs mini-SWE-agent (Yang et al., 2024a) on the Lean problems of PutnamBench (Tsoukalas et al., 2024). Among the trajectories that finish without errors within 128K tokens, we keep the 100 with the most output tokens, which contain 6,348 requests. It is a **decode-heavy** workload. Each request generates 2,292 tokens on average and up to 32,769 tokens, almost entirely reasoning, while the median prompt is 37K tokens. Each session replays one trajectory in a closed loop and sends the next request once the previous response arrives and the recorded tool time elapses. Each response has its recorded output length, and later prompts contain the outputs that the server generated, including their reasoning. Consecutive requests therefore share prefixes as in the original agent (Appendix C).

Models and deployment. We serve three hybrid models with different recurrent layers (Table 2). GLM-5.3-Flash (Z.ai, 2026) uses KDA, Qwen3.8-Flash-Next (Qiu et al., 2026) uses Gated DeltaNet, and Nemotron 3 Super (Chandiramani et al., 2026) uses Mamba2. Each engine runs on two NVIDIA B200 GPUs with tensor

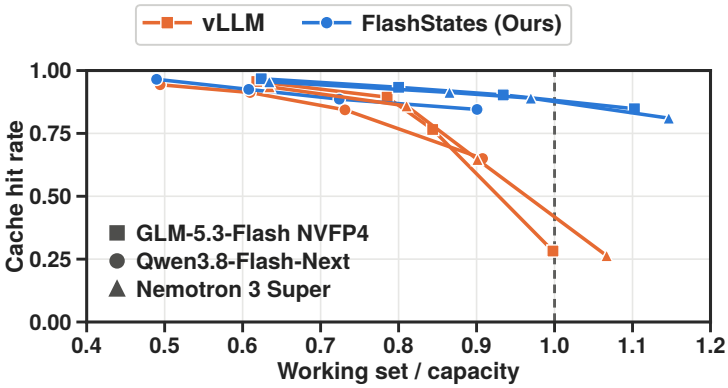


Figure 6 Cache hit rate versus working set. vLLM's cache hit rate collapses as its working set approaches the capacity, whereas FlashStates keeps a high cache hit rate beyond it. Each point represents a 75-minute working-set measurement.

parallelism. The coding workload uses a 1M-token context for GLM-5.3-Flash and Nemotron 3 Super and a 512K-token context for Qwen3.8-Flash-Next. The math workload uses GLM-5.3-Flash with 128K context.

Baseline. We compare FlashStates with vLLM (Kwon et al., 2023). For hybrid models, vLLM saves a recurrent checkpoint at every KV block boundary within the prompt and evicts cached blocks in LRU order (Section 2). We implement FlashStates in vLLM. Both systems use the same cache budget, batch size, and prefill chunk size, so their differences stem only from hybrid state management. Section 2 compares vLLM with SGLang (Zheng et al., 2024). Appendix D shows that FlashStates is compatible with host-memory offloading.

Evaluation settings. We vary the number of concurrent sessions from 24 to 48 for the coding workload and from 16 to 64 for the math workload, both in steps of 8. When a session finishes, the next trajectory starts in its place. For GLM in Figures 1b and 5, we measure the final 30 minutes of 45-minute runs after warm-up. The other main experiments use 75-minute runs, measured over their final 60 minutes, with the engine, warm-up, and statistical settings in Appendix C. vLLM and FlashStates run on separate GPUs; their clients can start at different wall-clock times, so we align their measurement windows by elapsed minute of each run. We treat a throughput difference as significant when it is at least twice its standard error. Where a point has two runs, figures report their mean. FlashStates keeps its configuration fixed across concurrency levels within each experiment. Its prefill cost $C(a, b)$ (Section 4.1) is fitted once per model from measured prefill times on B200 GPUs (Appendix C).

Metrics. *Decode throughput* counts the output tokens generated per minute per GPU. *Prefill throughput* counts the prompt tokens of requests whose first output token arrives in the measurement window, per minute per GPU. It includes both the tokens read from the cache and the tokens computed, so a system that recomputes less serves more prompts in the same time. *Uncached tokens per request* counts the prompt tokens that the engine computes rather than reads from the cache, and we report its median. *p90 TTFT* is the 90th-percentile time to first token. *Cache hit rate* is the fraction of prompt tokens read from the cache.

5.2 Coding Agent Serving Performance

FlashStates sustains higher throughput than vLLM as concurrency grows on all three models (Figure 5). Its peak decode throughput is 1.14–1.28× that of vLLM and occurs about 8 sessions later. Beyond its peak, the cache hit rate of vLLM collapses, so its requests recompute most of their prompts, which delays their first tokens and takes GPU time from decoding (Figure 6). FlashStates keeps its cache hit rate high, so its throughput stays near its peak up to 48 sessions.

The cache hit rates of the two systems diverge once the working set approaches the cache capacity. We define the working set as the cache blocks that every live conversation, including sub-agents, needs to resume its next request without recomputation, namely the attention KV of its latest context and a recurrent checkpoint at its end. Because agent contexts are long, the working set estimated from the trace statistics reaches the

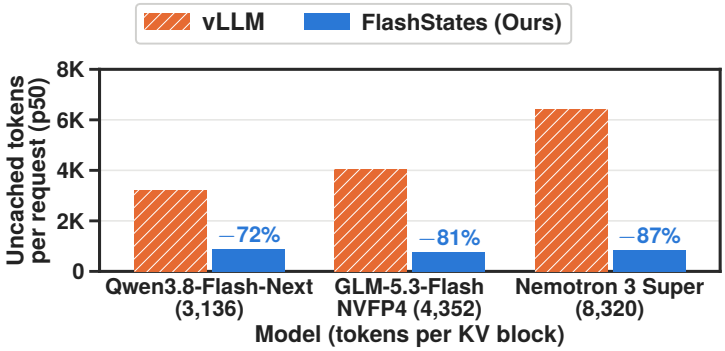


Figure 7 The reduction in uncached tokens grows with the KV block size. Median uncached tokens per request at 24 sessions, where the cache holds the working set. Labels give FlashStates's reduction relative to vLLM.

cache capacity at only 32–49 sessions (Figure 10 in Appendix C). While the working set fits in the cache, both systems achieve similar throughput, but FlashStates recomputes fewer prompt tokens per request, reducing the prefill work before the first token (Section 5.3). Beyond that point, the admission and eviction policy decides which conversations resume without recomputation. vLLM evicts blocks by recency regardless of state–KV dependencies, so it loses checkpoints whose KV remains cached (Section 2.2). FlashStates applies the same dependency-aware GDS priority when admitting a newly cached hybrid state under pressure and when evicting resident hybrid states (Section 4.3), so it keeps a high cache hit rate even beyond the cache capacity.

5.3 Versatility across Hybrid Architectures

FlashStates improves serving on all three hybrid architectures shown in Figure 5. The three models use different recurrent layers, and vLLM stores their KV in blocks of 3,136 to 8,320 tokens (Table 2).

The reduction in uncached tokens per request grows with the KV block size (Figure 7). Tokens between the actual and ideal resume positions of a request must be recomputed (Section 4.1). In an agent session, the ideal resume position of a request is usually the end of the previous request, because the new prompt extends the previous prompt with its output (Appendix C; Proposition 1). vLLM saves recurrent checkpoints only at block boundaries within the prompt, so its actual resume position is the last block boundary of the previous prompt. The gap between the two positions covers the rest of the previous prompt and the previous output, and it grows with the block size (Figure 1a). On every model, FlashStates lowers the median uncached tokens per request by about 70% of a block relative to vLLM. FlashStates saves a recurrent checkpoint at the exact end of each request, so its actual resume position reaches the ideal one whenever that checkpoint and its KV remain cached, regardless of the block size. When a request completes, the engine holds what a recurrent checkpoint at its end requires (Section 3.1; Appendix G), so the same mechanism applies to all three architectures. This result supports our key insight that resume positions should be determined by opportunities for prefix reuse rather than by physical KV allocation (Section 3).

5.4 Math Research Agent Serving Performance

FlashStates also improves serving of the math research agent (Figure 8). Its decode throughput is 2.0–9.6% higher than that of vLLM at 16–64 sessions. Because the math trajectories are short and their requests decode long outputs, the live conversations do not contend for the cache, and the cache hit rate of vLLM stays flat as concurrency grows.

The gain therefore comes from resume positions rather than from admission and eviction. Each request generates a long reasoning output, and the next prompt keeps it. vLLM saves no recurrent checkpoint during decoding (Section 2.1), so its actual resume position lies before the previous output, and it recomputes that whole output on every turn. FlashStates resumes from the ideal resume position at the exact end of the previous request and recomputes at most about 1% of the previous output. Because recomputation competes

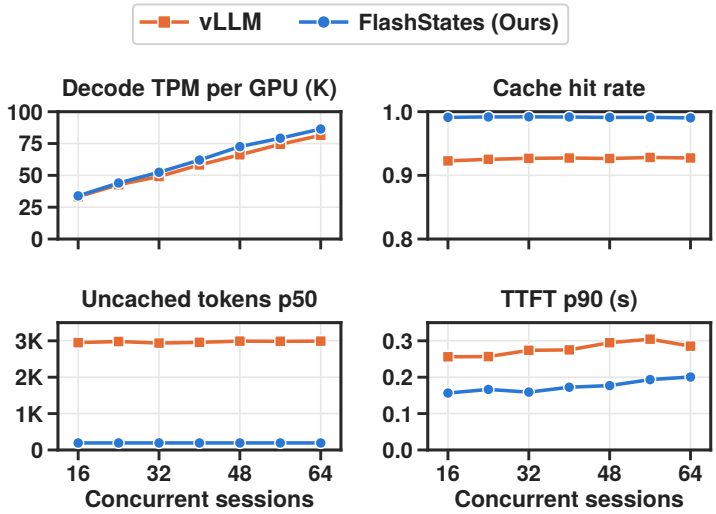


Figure 8 Serving performance on the math research agent. vLLM and FlashStates serve the math workload with GLM-5.3-Flash and a 128K-token context, and each point is the mean of two runs. FlashStates keeps a higher cache hit rate and fewer uncached tokens per request at every load, which raises its decode throughput and lowers its time to first token.

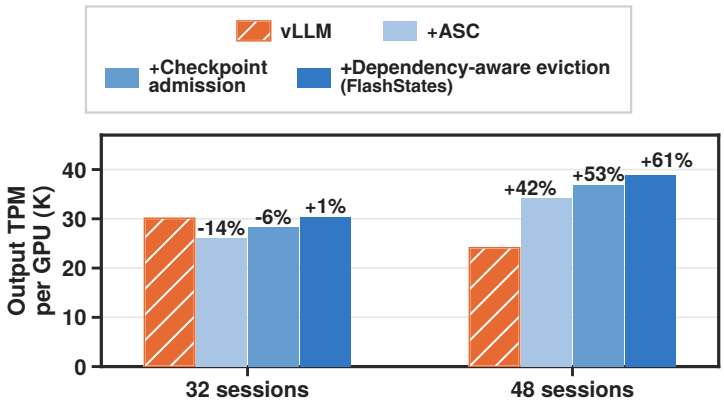


Figure 9 Performance gain from each stage of FlashStates. Performance of serving Qwen3.8-Flash-Next (512K context); each bar adds ASC, checkpoint admission, then dependency-aware eviction. Labels give the change from vLLM.

with decoding for the GPU, the saved prefill also speeds up decoding, and each sequence of FlashStates decodes 1–9% faster than under vLLM as concurrency grows.

5.5 Ablation Study

We add the stages of FlashStates to vLLM one at a time and run all four configurations together on Qwen3.8-Flash-Next 512K at 32 and 48 sessions (Figure 9). The first stage adds ASC, which saves a recurrent checkpoint at every prompt end and output end, and keeps the LRU eviction of vLLM. Checkpoint admission adds dependency-aware hybrid-state units with an LRU score, and dependency-aware GDS eviction replaces this score to complete FlashStates (Section 4.3).

Figure 9 shows that ASC and hybrid state management complement each other. At 48 sessions, near the cache capacity, ASC with the LRU eviction of vLLM already provides about 70% of the gain of FlashStates over vLLM, and checkpoint admission and dependency-aware eviction add 7.6% and 5.3% (Table 3). At 32 sessions, the working set uses only about 60% of the cache capacity (Figure 10), yet ASC with LRU eviction falls below vLLM in decode throughput. LRU keeps every checkpoint that ASC saves, although a conversation usually resumes from its latest one, except after a context edit (Appendix E). The older checkpoints then push out the context of conversations that wait for long tool calls (Appendix C). Checkpoint admission and

dependency-aware eviction instead choose which hybrid states to keep, adding 8.5% and 7.8% and bringing the full stack to the level of vLLM. The additional positions that ASC exposes therefore pay off reliably only when admission and eviction choose among them by their dependencies and priorities.

6 Conclusion

We present FlashStates, a serving system that enables flexible hybrid state management. When serving coding-agent workloads with GLM-5.3-Flash, Nemotron 3 Super and Qwen3.8-Flash-Next on NVIDIA B200 GPUs, FlashStates achieves up to 3.14×, 2.27× and 1.54× the prefill throughput of vLLM respectively.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.298. https://aclanthology.org/2023.emnlp-main.298/.

AIPerf. AIPerf source code, main branch as of 11 October 2026. https://github.com/ai-dynamo/aiperf, 2026. Agentic trace replay and the AgentX scenario.

Mayank Arya, Agrim Bari, Anjaly Parayil, Yogesh Simmhan, Renee St. Amant, Victor Rühle, and Saravan Rajmohan. Understanding KV residency in production-scale agentic LLM workloads. *arXiv preprint arXiv:2610.22863*, 2026. https://arxiv.org/abs/2610.22863.

Malika Aubakirova, Alex Atallah, Chris Clark, Justin Summerville, and Anjney Midha. State of AI: An empirical 100 trillion token study with OpenRouter. *arXiv preprint arXiv:2601.10088*, 2026. https://arxiv.org/abs/2601.10088.

Pei Cao and Sandy Irani. Cost-Aware WWW proxy caching algorithms. In *USENIX Symposium on Internet Technologies and Systems (USITS 97)*, Monterey, CA, December 1997. USENIX Association. https://www.usenix.org/conference/usits-97/cost-aware-www-proxy-caching-algorithms.

Aakshita Chandiramani, Aaron Blakeman, Abdullahi Olaoye, Abhibha Gupta, Abhilash Somasamudramath, Abhinav Khattar, Adeola Adesoba, Adi Renduchintala, Adil Asif, Aditya Agrawal, et al. Nemotron 3 Super: Open, efficient mixture-of-experts hybrid Mamba-Transformer model for agentic reasoning. *arXiv preprint arXiv:2604.12374*, 2026. https://arxiv.org/abs/2604.12374.

Chaokun Chang, Yukun Zhou, Kaihua Fu, Dakai An, Tianyu Feng, Hanfeng Lu, Sheng Yao, Pu Guo, Yinghao Yu, Yizhou Shan, Bo Li, Binhang Yuan, and Wei Wang. From LLM inference to agentic workloads: Characterization and implications for serving systems. *arXiv preprint arXiv:2608.15127*, 2026. https://arxiv.org/abs/2608.15127.

Hao Che, Ye Tung, and Zhijun Wang. Hierarchical web caching systems: Modeling, design and experimental results. *IEEE Journal on Selected Areas in Communications*, 20(7):1305–1314, 2002. doi: 10.1109/JSAC.2002.801752.

Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 10041–10071, 2024.

DeepSeek-AI, Aixin Liu, Aoxue Mei, et al. DeepSeek-V3.2: Pushing the frontier of open large language models. arXiv preprint arXiv:2512.02556, 2025. https://arxiv.org/abs/2512.02556.

Mostafa Dehghan, Laurent Massoulié, Don Towsley, Daniel Sadoc Menasché, and Yong Chiang Tay. A utility optimization approach to network cache design. *IEEE/ACM Transactions on Networking*, 2019. doi: 10.1109/TNET.2019.2913677.

Andrés Ferragut, Ismael Rodríguez, and Fernando Paganini. Optimizing TTL caches under heavy-tailed demands. In *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Science*, 2016. doi: 10.1145/2896377.2901459.

Christine Fricker, Philippe Robert, and James Roberts. A versatile and accurate approximation for LRU cache performance. In *Proceedings of the 24th International Teletraffic Congress (ITC)*, 2012.

Dongwook Kang. Native dual pools or shared paging? memory organization for concurrent hybrid LLM serving. arXiv preprint arXiv:2610.15755, 2026. https://arxiv.org/abs/2610.15755.

Hao Kang, Ziyang Li, Weili Xu, Xinyu Yang, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. ThunderAgent: A fast, simple, and program-aware agentic inference system. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 55630–55657. PMLR, 2026. https://proceedings.mlr.press/v306/kang26j.html.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 5156–5165, 2020.

Kimi Team, Yu Zhang, Zongyu Lin, et al. Kimi Linear: An expressive, efficient attention architecture. arXiv preprint arXiv:2510.26692, 2025. https://arxiv.org/abs/2510.26692.

Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, M. C., Jianfeng Cai, Xinyuan Cai, Peizhou Cao, Yuxuan Cao, Ziwei Chai, Y. Charles, et al. Kimi K3: Open frontier intelligence. *arXiv preprint arXiv:2607.24653*, 2026. https://arxiv.org/abs/2607.24653.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023. doi: 10.1145/3600006.3613165.

Martin A. Lariviere. A note on probability distributions with increasing generalized failure rates. *Operations Research*, 2006. doi: 10.1287/opre.1060.0282.

Ze-Wei Liou, Harris Nover, Amey Naik, Andrii Skliar, Igor Shovkun, Ido Shahaf, and Tri Dao. ReplaySSM: Cache SSM inputs, not state. arXiv preprint 2610.13778, 2026.

Rui Pan, Zhuang Wang, Zhen Jia, Can Karakus, Luca Zancato, Tri Dao, Yida Wang, and Ravi Netravali. Marconi: Prefix caching for the era of hybrid LLMs. In *Proceedings of Machine Learning and Systems 7 (MLSys)*, 2025.

Zaifeng Pan, Lucas Wang, Xinwei Qiang, Yanbo Zhou, Chang Chen, Zhengding Hu, Yue Guan, and Yufei Ding. HiState: Hierarchical prefix cache management for hybrid LLM serving in agentic workloads. *arXiv preprint arXiv:2610.41508*, 2026. https://arxiv.org/abs/2610.41508.

Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *ACM Transactions on Storage*, 2025. doi: 10.1145/3773772.

Zihan Qiu, Zekun Wang, Xiao Li, Yanpeng Li, Yang Xu, Yixuan Wang, Huaqing Zhang, Rui Men, Bochao Mao, Chengruidong Zhang, et al. On the design of Qwen3.8-Next architecture: Evaluation, efficiency, and training stability. *arXiv preprint arXiv:2608.30320*, 2026. https://arxiv.org/abs/2608.30320.

SemiAnalysis. CC traces: Weka, with subagents, v7 only. Hugging Face dataset, https://huggingface.co/datasets/semianalysisai/cc-traces-weka-062126, June 2026.

SGLang. [Prefix Cache / Hybrid] Branch reuse silently collapses under LRU pressure when (kv_pool_tokens / chunked_prefill_size) > max_mamba_cache_size. SGLang issue 36935 (open), https://github.com/sgl-project/sglang/issues/36935, 2026a.

SGLang. SGLang source code, release v0.5.22. https://github.com/sgl-project/sglang, 2026b. `python/sglang/srt/arg_groups/model_overrides/deepseek_v2.py` (page size for sparse-attention models), `python/sglang/srt/arg_groups/fields/schedule.py` (`mamba_full_memory_ratio`), `python/sglang/srt/arg_groups/overrides.py` (`extra_buffer`), `python/sglang/srt/managers/io_struct.py` (sessions).

SGLang Team. SGLang server arguments, 2026a. https://docs.sglang.io/docs/advanced_features/server_arguments. Online documentation; accessed 2026-10-28.

SGLang Team. SGLang unified radix cache components, 2026b. https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/mem_cache/unified_cache/components/README.md. Source documentation; accessed 2026-10-28.

Mikhail Shirokikh and Sergey Nikolenko. Sparse prefix caching for hybrid and recurrent LLM serving. arXiv preprint arXiv:2605.05219, 2026. https://arxiv.org/abs/2605.05219.

George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the Putnam mathematical competition. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*, pages 11545–11569, 2024. https://proceedings.neurips.cc/paper_files/paper/2024/hash/1582eaf9e0cf349e1e5a6ee453100aa1-Abstract-Datasets_and_Benchmarks_Track.html.

vLLM. [Hybrid KV Cache] Retain finalized Mamba decode checkpoints for prefix caching. vLLM pull request 50551 (open), https://github.com/vllm-project/vllm/pull/50551, 2026a.

vLLM. [Bugfix][Core] Retain align-mode mamba grid and decode-end checkpoints. vLLM pull request 53803 (open), https://github.com/vllm-project/vllm/pull/53803, 2026b.

vLLM. vLLM source code, main branch as of 14 October 2026. https://github.com/vllm-project/vllm, 2026c. `vllm/platforms/interface.py` (attention block enlarged to the Mamba page size), `vllm/v1/core/sched/scheduler.py` (block-aligned split; fine-grained hits), `vllm/config/cache.py` (`prefix_match_unit`, `kv_offloading_size`).

vLLM Project. vLLM hybrid KV cache manager, 2026. https://docs.vllm.ai/en/latest/design/hybrid_kv_cache_manager/. Online documentation; accessed 2026-10-28.

Zhiqiang Xie. SGLang HiCache: Fast hierarchical KV caching with your favorite storage backends. LMSYS Org blog, https://lmsys.org/blog/2025-09-10-sglang-hicache, 2025. September 10, 2025.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024a. https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

Shuo Yang, Xiaoze Fan, Melissa Pan, Haocheng Xi, Zhe Wang, Shanlin Sun, Kurt Keutzer, Song Han, Matei Zaharia, Chenfeng Xu, and Ion Stoica. FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution. arXiv preprint arXiv:2608.16157, 2026. https://arxiv.org/abs/2608.16157.

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024b.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. https://openreview.net/forum?id=WE_vluYUL-X.

Neal E. Young. On-line file caching. *Algorithmica*, 33(3):371–383, 2002. doi: 10.1007/s00453-001-0124-5.

Yichao Yuan, Ankita Nayak, Souvik Kundu, and Nishil Talati. Agentic AI workload characteristics. In *IEEE International Symposium on Workload Characterization (IISWC)*, 2026. https://arxiv.org/abs/2605.26297.

Z.ai. GLM-5.3-Flash: Frontier intelligence, flash cost, August 2026. https://z.ai/blog/glm-5.3-flash. Published August 26, 2026; accessed October 29, 2026.

Chen Zhang, Kuntai Du, Shu Liu, Woosuk Kwon, Xiangxi Mo, Yufeng Wang, Xiaoxuan Liu, Kaichao You, Zhuohan Li, Mingsheng Long, Jidong Zhai, Joseph Gonzalez, and Ion Stoica. Jenga: Effective memory management for serving LLM with heterogeneity. arXiv preprint arXiv:2503.18292, 2025. https://arxiv.org/abs/2503.18292.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. TraceLab: Characterizing coding agent workloads for LLM serving. *arXiv preprint arXiv:2606.30560*, 2026. https://arxiv.org/abs/2606.30560.

A Terminology

In the linear-attention literature, the recurrent state of a layer denotes the live mathematical quantity updated at every token, such as accumulated outer products of key and value vectors (Yang et al., 2024b, 2025); state-space models keep an analogous fixed-size state. This paper reserves *recurrent checkpoint* for a materialized snapshot of all recurrent-layer states at one exact prefix position. For a fixed model and runtime configuration, its size is independent of prefix length, and a recurrent checkpoint at a later position cannot be truncated to recover an earlier one (Pan et al., 2025).

The *hybrid state* of an exact prefix is the complete reusable inference state at its endpoint: the attention KV for the entire prefix and the recurrent checkpoint at that endpoint. Implementations also keep matching model-specific auxiliary state with it, such as convolution buffers or indexer history. Hybrid state is a logical recovery unit rather than one contiguous allocation: its components may occupy separate cache allocations, and its KV-cache blocks may be shared with other hybrid states.

This paper uses *KV-cache block* for vLLM's fixed-size cache-allocation unit, whether it holds attention KV or a recurrent-checkpoint shard. A complete recurrent checkpoint may span several same-sized KV-cache blocks across recurrent cache groups. The KV-cache block size in tokens and a KV-cache block boundary refer to the attention-KV layout; cache capacity counts all resident blocks regardless of their contents.

We distinguish three decisions. *Recurrent-checkpoint placement* determines the prefix positions where a recurrent checkpoint can be materialized; ASC changes placement. *Hybrid-state admission* decides whether newly materialized hybrid state remains cached. *Hybrid-state eviction* is the generic decision to remove cached hybrid state under pressure, and dependency-aware GDS eviction is FlashStates's concrete policy.

B Motivation Experiment Details

Experiment Setup. The profiling study in Section 2 serves GLM-5.3-Flash (NVFP4) on two NVIDIA B200 GPUs with tensor parallelism. Each concurrent session replays a coding-agent trajectory (SemiAnalysis, 2026) truncated at 128K tokens, and keeps the reasoning tokens of earlier turns in later prompts. We run vLLM at commit ec8a372 (0.32.0rc1.dev105), a development build shortly before v0.32.0 and the same build as in Section 5 (Appendix C), and SGLang v0.5.22 (released October 19, 2026), each with its default configuration and the same cache budget of 62.4 GiB per GPU. vLLM stores KV in 4,352-token blocks, while SGLang uses 256-token pages and splits memory between a KV pool and a recurrent-state pool. We vary the number of concurrent sessions from 8 to 64 and measure each 45-minute run after a 10-minute warm-up.

Cache Miss Attribution. For each request, we compare its cached prefix with its ideal resume position, the end of the longest prefix that earlier requests have already computed. Logging-only instrumentation in each engine records every checkpoint save, eviction, and prefix match, and does not change any scheduling or eviction decision. These logs let us assign every uncached prompt token to exactly one cause. Besides the four causes in Section 2, a token can miss because another request is still computing its prefix, which accounts for at most 0.2% of uncached prompt tokens.

C Evaluation Details

Engine version. The vLLM baseline runs at commit ec8a372 (0.32.0rc1.dev105), a development build shortly before v0.32.0, and FlashStates is implemented in the same build.

Replay client. Our replay client builds on the AgentX trace-replay scenario of AIPerf (AIPerf). AgentX sends each request with its recorded prompt, so a later prompt contains the recorded outputs of earlier requests rather than the outputs that the served model generated. When the served model differs from the recorded one, the hybrid state that the server caches at the end of a request no longer matches the start of the next prompt, and the replay understates the reuse that a real agent offers. We extend the client to send prompts as token IDs and to replace each earlier output in a later prompt with the token IDs that the server returned for that request. In coding runs at 32 sessions, 96.9–98.1% of later requests then start with exactly the previous prompt followed by its generated output, and the rest follow context rewrites recorded in the trajectories

(Appendix E). The client rebuilds each prompt from 64-token hash blocks and drops a partial last block of the previous prompt. The coding trajectories are block-aligned by construction, and we align the prompt lengths of the math trajectories to these blocks. In the converted math trajectories, all 6,004 later requests that keep the previous output start with exactly the previous prompt and that output.

Replay settings. Each request sets its maximum output length to the recorded length and ignores end-of-sequence tokens, so both systems generate the same number of tokens. The client waits the recorded tool time between a response and the next request, and caps each period in which a trajectory has no request in flight at 300 s. Sub-agents run as separate conversations in their recorded spawn and join order. Later prompts keep the reasoning tokens of earlier outputs. Each session starts at a random point between 25% and 75% of its trajectory with a fixed seed. In the main experiments, a warm-up phase sends 10 requests per session before measurement.

Prefill cost model. FlashStates estimates the prefill time over positions $[a, b)$ (Section 4.1) as $C(a, b) = d\,(b - a) + e\,(b^2 - a^2)/(2 \cdot 10^5)$ seconds, whose second term grows with the attention context. For GLM-5.3-Flash and Qwen3.8-Flash-Next, we fit d and e by least squares to the prompt-processing time of earlier serving runs on two B200 GPUs. For Nemotron 3 Super, we fit them to 16 prefills of 128K–960K tokens on vLLM. The fitted values are $(d, e) = (1.862, 0.333) \times 10^{-5}$ for GLM-5.3-Flash, $(2.230, 0.434) \times 10^{-5}$ for Qwen3.8-Flash-Next, and $(1.322, 0.419) \times 10^{-5}$ for Nemotron 3 Super. The parameters stay fixed across workloads and concurrency levels.

Working set. A conversation is live from its first request until its last request finishes, and each sub-agent counts as its own conversation. Its context is the prompt and output of its most recent completed request, or the prompt of its first request while that request runs. A context of L tokens needs $\lceil L/B \rceil$ blocks of attention KV for block size B, and its recurrent checkpoint, together with the auxiliary state, needs five blocks on each of the three models. We sample the measured working set every second, average it per minute, and report its median over the measurement window, relative to the 2,531, 3,332, and 1,415 blocks of the cache of GLM-5.3-Flash, Qwen3.8-Flash-Next, and Nemotron 3 Super. The trace estimate multiplies the number of sessions, the number of live conversations per session measured in our runs, and the blocks of one conversation, with L set to the mean context per request for the lower edge of the band and to the mean final context per trajectory for the upper edge. Figure 10 compares this estimate with the measured working set.

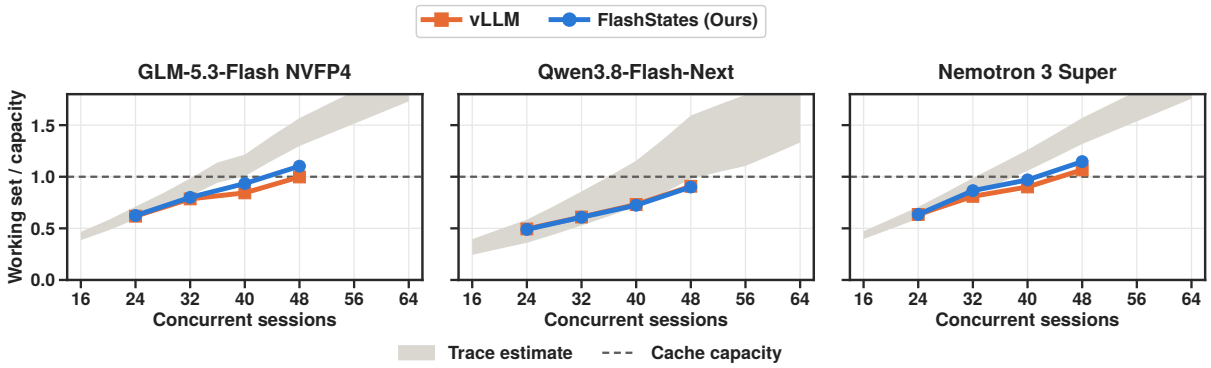


Figure 10 Working set across concurrency. The band estimates the working set from the trace statistics, between the mean context per request and the mean final context per trajectory. The lines give the median working sets measured in the 75-minute profiling runs.

Statistics. For the main serving experiments, we split the measurement window of each run into one-minute bins and compute the per-minute difference between FlashStates and vLLM. The standard error comes from the means of twelve five-minute batches, and a difference counts as significant when it is at least twice its standard error. For points with two runs, we pool the two differences by inverse variance.

Ablation. Table 3 reports the metrics of the ablation in Section 5.5. Each load has one run, and the four configurations of a run start within 3 s of each other. Relative to vLLM, the decode throughput of ASC alone is significantly lower at 32 sessions and significantly higher at 48 sessions. At 32 sessions, neither

Table 3 Ablation metrics. Qwen3.8-Flash-Next with a 512K-token context. Each configuration adds the next method stage to the row above it, and the last row is FlashStates. Decode and prefill throughput are per GPU.

Configuration	Decode TPM per GPU (K)	Prefill TPM per GPU (M)	Cache hit rate	Uncached tokens p50	TTFT p90 (s)
32 sessions					
vLLM	30.1	3.37	0.912	3,377	0.83
+ASC	26.0	2.96	0.847	1,025	1.93
+Checkpoint admission	28.2	3.17	0.889	961	1.01
+Dependency-aware eviction (FlashStates)	30.4	3.40	0.925	1,089	0.75
48 sessions					
vLLM	24.1	2.36	0.646	4,323	7.54
+ASC	34.2	3.36	0.809	1,153	2.47
+Checkpoint admission	36.8	3.59	0.833	1,217	1.72
+Dependency-aware eviction (FlashStates)	38.8	3.74	0.847	1,473	1.45

the checkpoint-admission stage nor FlashStates differs significantly from vLLM. At 48 sessions, both are significantly higher than vLLM. Relative to the preceding stage, checkpoint admission adds 8.5% at 32 sessions and 7.6% at 48 sessions; both differences are significant. Dependency-aware eviction then adds 7.8% and 5.3%, respectively. At 32 sessions with ASC alone, requests that find less than 10% of their reusable context in the cache recompute 39.8M tokens beyond their ideal resume positions, compared with 9.6M under vLLM. Of these tokens, 37.1M belong to requests sent at least 60 s after the previous request of the same conversation ended. Checkpoint admission reduces the 39.8M tokens to 25.7M, and dependency-aware eviction reduces them further to 10.6M.

Larger GPU cache. The main Nemotron 3 Super experiment uses a 22.46 GiB cache per GPU, selected to provide approximately 10.45M tokens of capacity as reported by vLLM. To test whether FlashStates's gains persist with a larger cache, we allocate 60 GiB per GPU to both engines, 2.67× the original budget and approximately 27.91M reported tokens. Each engine runs on two B200 GPUs with tensor parallelism and a 1M-token context limit, without host offloading or speculative decoding. Using the AgentX coding traces, we report 17 concurrency levels from 32 to 96 sessions in steps of four. This supplementary experiment uses the FlashStates release implementation with an updated replay protocol, measuring the final 30 minutes of a 45-minute run at each load.

Figure 11 shows that at 32–64 sessions, both engines keep p90 TTFT below one second, while FlashStates's decode throughput is 1.2–11.1% higher. At 84–96 sessions, FlashStates delivers 1.59–2.67× vLLM's decode throughput. At 96 sessions, it serves 50.5K versus 18.9K output tokens per minute per GPU, with p90 TTFT of 1.40 s versus 28.86 s. Its prefix hit rate is also higher at this load, 85.0% versus 51.8%, consistent with preserving more reusable context. These results show that FlashStates's throughput and latency gains persist with a substantially larger GPU cache as concurrency grows.

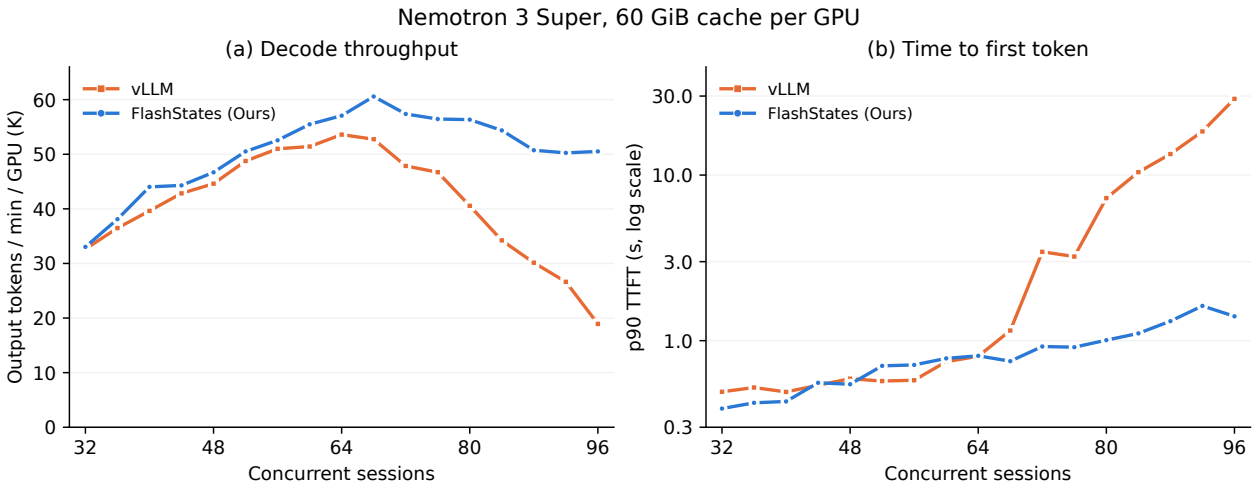


Figure 11 FlashStates retains its gains with a 60 GiB GPU cache per GPU on Nemotron 3 Super. Results are shown for the 17 paired concurrency levels from 32 to 96 sessions. Each point reports one valid run per engine: a 45-minute profiling phase following warm-up, measured over minutes 15–45. Decode throughput counts output tokens by stream-chunk arrival and is divided by two GPUs; p90 TTFT covers requests sent in the measurement window. The TTFT axis is logarithmic. Lines connect measured points; no uncertainty across repeated runs is estimated.

D FlashState with CPU Caching and Shared Storage

We test whether FlashStates's gains under GPU-cache pressure persist with approximately 512 GiB of CPU cache per service. A larger cache retains more data, but **cannot provide a recurrent checkpoint that was never captured**. We compare vLLM and FlashStates with 512 GiB Mooncake stores against SGLang + HiCache (Zheng et al., 2024; Xie, 2025) (SGLang v0.5.23) with **a single 511.5 GiB host tier and no Mooncake backend**. All three serve GLM-5.3-Flash NVFP4 on two B200 GPUs with multi-token prediction (MTP) enabled for three speculative tokens. We replay the Claude Code trajectories released by SemiAnalysis (SemiAnalysis, 2026) at all 49 integer loads from 32 to 80 sessions for the Mooncake stacks, and at seven loads in steps of eight for SGLang + HiCache.

FlashStates improves output throughput over vLLM by 2.1–41.4% across the 49 loads and **over SGLang + HiCache by 3.2–36.2%** at the seven shared loads (Table 4). SGLang + HiCache has lower p90 TTFT at 32–48 sessions, while FlashStates has lower p90 TTFT at 56–80. At 80 sessions, vLLM, SGLang + HiCache, and FlashStates respectively deliver 122.2K, 126.9K, and **172.8K** output tokens per minute per service, with p90 TTFT of 21.88, 22.97, and **7.18 s**. Against vLLM, median uncached prompt tokens fall by 68.7–81.4%; exact-endpoint reuse avoids reprocessing the previous turn's suffix even when its earlier prefix is cached. The three-way comparison evaluates **complete serving stacks**, whose GPU memory layouts and data-transfer paths also differ.

D.1 FlashStates's Mooncake Store

FlashStates changes how hybrid states enter the Mooncake store and how the store evicts them. With **asynchronous request-end write-through**, FlashStates queues the hybrid state at the exact processed endpoint of each finished request for GPU-to-CPU transfer when the request completes, without waiting for GPU eviction. Stock Mooncake evicts individual objects, oldest lease first, which approximates least-recently-used (LRU) order, and it does not know which objects belong to the same hybrid state. We extend Mooncake's metadata server to track state ownership and shared KV references. The store then evicts each cached hybrid state as one *logical recovery unit*, namely its recurrent checkpoint and the KV pages that no other cached hybrid state requires, and it ranks the units by Section 4's dependency-aware GDS priority instead of recency. Pages that other hybrid states still require stay in the store.

After writes complete, a later turn on another engine can reuse the state only with an exact prefix match

Table 4 FlashStates retains throughput gains with approximately 512 GiB of CPU cache and MTP. The table compares all seven loads measured for SGLang + HiCache with the corresponding vLLM + Mooncake and FlashStates runs. In the table, vLLM denotes vLLM + Mooncake, SGLang denotes SGLang + HiCache, and FlashStates runs with its modified Mooncake store (Appendix D.1). FlashStates rows are blue and bold. Values in parentheses compare SGLang and FlashStates with vLLM, as a ratio for throughput and decode speed and as a change for TTFT and uncached tokens, where negative is better.

Concurrent agents	System	Prefill TPM (M/GPU)	Output TPM (K)	TTFT p50 (s)	TTFT p90 (s)	Decode p50 (tokens/s)	Uncached tokens p50
32	vLLM	5.76	77.9	0.28	0.80	151.8	3,454.5
	SGLang	5.63 (×0.98)	77.0 (×0.99)	0.14 (−50.0%)	0.43 (−46.2%)	145.9 (×0.96)	803 (−76.8%)
	FlashStates	**6.12** (×1.06)	**79.5** (×1.02)	**0.18** (−35.7%)	**0.64** (−20.0%)	**177.3** (×1.17)	**641** (−81.4%)
40	vLLM	6.98	95.0	0.30	0.88	124.1	3,443.5
	SGLang	6.64 (×0.95)	92.8 (×0.98)	0.15 (−50.0%)	0.52 (−40.9%)	115.3 (×0.93)	899 (−73.9%)
	FlashStates	**7.49** (×1.07)	**103.1** (×1.09)	**0.19** (−36.7%)	**0.65** (−26.1%)	**153.9** (×1.24)	**705** (−79.5%)
48	vLLM	8.13	105.4	0.38	1.28	103.9	3,398
	SGLang	7.88 (×0.97)	102.8 (×0.98)	0.17 (−55.3%)	0.74 (−42.2%)	102.8 (×0.99)	948 (−72.1%)
	FlashStates	**8.81** (×1.08)	**114.5** (×1.09)	**0.23** (−39.5%)	**0.92** (−28.1%)	**132.4** (×1.27)	**833** (−75.5%)
56	vLLM	8.63	122.3	0.70	2.24	83.9	3,482
	SGLang	8.41 (×0.97)	119.8 (×0.98)	0.22 (−68.6%)	3.07 (+37.1%)	88.3 (×1.05)	1,038.5 (−70.2%)
	FlashStates	**9.58** (×1.11)	**137.0** (×1.12)	**0.28** (−60.0%)	**1.11** (−50.4%)	**110.5** (×1.32)	**960** (−72.4%)
64	vLLM	8.84	129.3	1.66	5.99	72.4	3,476
	SGLang	8.92 (×1.01)	129.2 (×1.00)	1.43 (−13.9%)	8.47 (+41.4%)	77.3 (×1.07)	1,060.5 (−69.5%)
	FlashStates	**10.18** (×1.15)	**153.0** (×1.18)	**0.40** (−75.9%)	**1.68** (−72.0%)	**96.0** (×1.33)	**960** (−72.4%)
72	vLLM	8.37	124.7	6.52	14.25	61.4	3,506
	SGLang	8.73 (×1.04)	127.8 (×1.02)	6.54 (+0.3%)	15.62 (+9.6%)	72.1 (×1.17)	1,125 (−67.9%)
	FlashStates	**10.96** (×1.31)	**166.6** (×1.34)	**0.86** (−86.8%)	**3.86** (−72.9%)	**78.9** (×1.29)	**1,024** (−70.8%)
80	vLLM	7.99	122.2	12.40	21.88	57.9	3,486
	SGLang	8.47 (×1.06)	126.9 (×1.04)	10.89 (−12.2%)	22.97 (+5.0%)	68.7 (×1.19)	1,166.5 (−66.5%)
	FlashStates	**11.18** (×1.40)	**172.8** (×1.41)	**2.34** (−81.1%)	**7.18** (−67.2%)	**70.7** (×1.22)	**1,025** (−70.6%)

Each service uses two B200 GPUs. Host-cache capacity is 512 GiB for each Mooncake stack and 511.5 GiB for SGLang + HiCache; GPU cache pools are 55.0, 54.0, and 60.7 GB per GPU for vLLM, FlashStates, and SGLang, respectively. vLLM uses the stock Mooncake client; FlashStates includes optimized GPU-to-CPU copies and compact speculative states. Each configuration has one independent 2,700-s run per load, measured over [900,2700) s; the SGLang + HiCache and Mooncake runs were collected separately using the same replay client and measurement protocol. Prefill TPM includes cached and computed prompt tokens, counted when the first output token arrives in the window and divided by two GPUs. Output TPM counts tokens arriving in the window per two-GPU service. Request quantiles use requests sent in the window; per-request decode speed is (output tokens − 1) / (completion time − first-token time), excluding TTFT. Uncached tokens include new input.

and all required objects across tensor-parallel ranks; otherwise, it recomputes. This supports reuse by a later prefill engine in future prefill–decode deployments, separately from the current request's prefill-to-decode handoff. We checked cross-engine reuse with two TP2 engines on one host that share a 16 GiB store, without speculative decoding. The engines resumed **all 52 continuation turns** at the previous processed endpoint, and the checked state and KV reads matched the saved values bit for bit.

E Preserving Exact Reuse across Agent Context Changes

After a context edit, the latest request-end checkpoint may no longer match, **while an earlier checkpoint remains useful**. When an agent edits a recent suffix, the earlier checkpoint may still match the unchanged prefix. In our 393 source coding-agent traces (SemiAnalysis, 2026), 3.85% of requests change the preceding input prefix.[2] Preserving reuse for these requests requires retaining a matching state before the edit.

We run FlashStates with the configuration of Figure 1b plus a 512 GiB Mooncake CPU store. Context

[2]We reconstruct conversations using the replay client and compare recorded input prefixes at 64-token hash-block resolution. Excluding initialization transitions, 3,808 of 98,827 source requests change the preceding input prefix. This audit does not check preservation of the previous generated output.

shortening is also infrequent in these runs, yet many shortened-context requests incur extra prefill. At 64 and 72 sessions, approximately 3% of requests shorten their context. When superseded states are immediately marked for preferential eviction, **55.1% and 58.7% of shortened-context requests** incur diagnosed extra prefill, corresponding to 1.63% and 1.72% of all requests (Table 5).

A newer checkpoint therefore need not make recent earlier checkpoints obsolete. The Mooncake connector delays marking superseded states for preferential eviction by two continuation turns, preserving recent request-end states and associated intermediate states. If an earlier checkpoint still matches, the engine resumes there and recomputes the remaining suffix (Figure 12). These states remain subject to capacity eviction; reuse requires an exact prefix match and complete hybrid state.

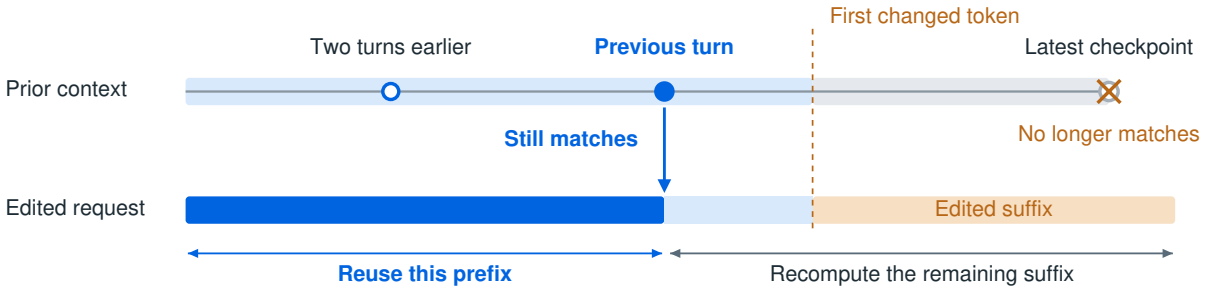


Figure 12 The latest checkpoint can stop matching while an earlier exact checkpoint remains useful. The edit falls after the previous checkpoint and before the latest one. FlashStates resumes from the previous checkpoint and recomputes the entire remaining suffix, including unchanged tokens between the checkpoint and the first edit. Reuse requires that the matching hybrid state remain intact; the two-turn delay does not exempt states from capacity eviction.

With this delay and the same 512 GiB host store, approximately 3% of requests still shorten their context, but the fraction of those requests with diagnosed extra prefill **falls to 16.2% and 18.2%**. Overall p90 TTFT falls by 18.1% and 19.4%. These results show that retaining recent exact states can preserve reuse across context changes and improve latency on this workload.

Table 5 Retaining recent exact states reduces observed reuse losses under context shortening. Both configurations use FlashStates + Mooncake with a 512 GiB host store and 66 GB of GPU cache per GPU; only the timing of marking superseded states for preferential eviction changes. Arrows give immediate marking → a two-turn delay (blue bold). Percentages, counts, and TTFT use the same request-send window [900,2700) s.

Sessions	Shortened / all	Extra prefill / shortened	Extra prefill / all	p90 TTFT (s)	TTFT reduction
64	2.95% → **2.89%** 98/3,319 → 99/3,425	55.1% → **16.2%** 54/98 → 16/99	1.63% → **0.47%** 54/3,319 → 16/3,425	1.88 → **1.54**	**18.1%**
72	2.93% → **3.04%** 104/3,548 → 110/3,617	58.7% → **18.2%** 61/104 → 20/110	1.72% → **0.55%** 61/3,548 → 20/3,617	5.78 → **4.65**	**19.4%**

Metric definitions. A shortened-context request has a prompt shorter than the preceding prompt plus output; this does not detect equal-length or longer edits. For shortened-context requests, extra prefill is diagnosed when cached-prefix reuse is below the largest reuse observed for the same logical request across both runs, capped at the current prompt length minus one token; losses shared by both runs are invisible. Counts under each percentage show its numerator and denominator. The two runs process different numbers of requests and are not paired event by event.

F Extended Related Work

This appendix compares FlashStates with other systems that retain recurrent checkpoints of hybrid models across requests. Table 6 summarizes the design choices of Marconi (Pan et al., 2025), the concurrent HiState (Pan et al., 2026), and FlashStates. We port Marconi into our vLLM build and measure it (Appendix F.1), and we compare with HiState by design (Appendix F.2). ReplaySSM (Liou et al., 2026) addresses a different

stage of hybrid-model serving. It represents the recurrent state during decoding as a checkpoint and a buffer of recent SSM inputs, which reduces state memory traffic and makes speculative rollback cheap. FlashStates instead decides which hybrid states remain cached across requests.

Table 6 Design choices of systems that retain recurrent checkpoints across requests. Each column follows the system's paper. Our port of Marconi in Appendix F.1 also saves a recurrent checkpoint at the exact end of each request.

	Marconi	HiState	FlashStates
Checkpoint positions	Branch points and the last decoded token	Adaptive intervals plus each segment's final state, at block granularity	Any token position (ASC)
Eviction unit	Cached prefix	Intermediate checkpoint, or last KV segment with its final checkpoint	Hybrid state with the allocations only it requires
Eviction score	Recency + $\alpha \times$ FLOPs saved per byte	Last-access time + $\tau \times$ recomputation per reclaimed storage	Watermark at last use + restore cost per released block
Weighting parameter	α, tuned online by replay	τ (time scale)	None

F.1 Comparison with Marconi

Marconi (Pan et al., 2025) evicts cached prefixes by a score that adds recency to FLOP efficiency, the computation a prefix saves per byte of memory, weighted by a parameter α. It tunes α once during serving. After its first eviction, it waits for five times as many requests as it has seen so far. It then replays the finished requests from an empty cache for every $\alpha \in \{0, 0.1, \ldots, 2.0\}$ and keeps the value that saves the most FLOPs over the last 500 of them. This section examines how serving throughput depends on α and whether this tuning finds a good value.

Setup. We port the eviction and tuning code of Marconi's official artifact into the same vLLM build as FlashStates. Our port also saves a recurrent checkpoint at the exact end of each request, so it differs from FlashStates only in checkpoint admission and eviction. We serve Qwen3.8-Flash-Next with a 512K-token context and **40 concurrent sessions**, with the settings of Section 5.1. We disable tuning and fix α at nine values from 0 to 2.0, and the nine configurations run together with FlashStates. In a separate run at the same load, the online tuner of our port selects $\alpha = 0$.

Table 7 Marconi with fixed α versus FlashStates. Qwen3.8-Flash-Next with a 512K-token context serves 40 concurrent sessions, and all ten configurations run together. Recompute is the number of computed prompt tokens relative to an ideal cache that computes only new content.

Configuration	Output TPM (K)	Prefill TPM per GPU (M)	Recompute ($\times$ ideal)	Uncached p50	TTFT p90 (s)
Marconi, $\alpha = 0$	66.0	3.40	2.96	1,025	1.84
Marconi, $\alpha = 0.1$	71.2	3.59	2.41	1,345	1.95
Marconi, $\alpha = 0.2$	68.0	3.45	2.84	1,729	2.32
Marconi, $\alpha = 0.3$	65.2	3.33	3.08	2,305	2.45
Marconi, $\alpha = 0.5$	63.1	3.18	3.44	2,888	3.03
Marconi, $\alpha = 0.7$	61.5	3.04	3.66	3,016	3.28
Marconi, $\alpha = 1.0$	59.9	2.97	3.74	3,201	3.22
Marconi, $\alpha = 1.5$	59.6	2.94	3.87	3,912	3.53
Marconi, $\alpha = 2.0$	59.9	2.97	3.82	3,905	3.48
FlashStates	73.4	3.64	2.06	1,217	1.05

Sensitivity to α. Throughput peaks at $\alpha = 0.1$ and falls as α grows beyond it (Table 7). Raising α from 0.1 to 0.2 already lowers output throughput by 4.8% $\pm$ 1.9%. Pooled over two runs, $\alpha = 1.0$ and $\alpha = 2.0$ serve 9.8% and 10.9% less than $\alpha = 0$, and both differences are significant.

Tuning. The tuner selects $\alpha = 0$ rather than the peak at 0.1. Its objective barely separates the candidates. Replaying the tuner on windows taken every 15 minutes of these runs selects 0 or 0.1 in every window, and $\alpha = 2.0$ saves at least 94% of the FLOPs of the best value. In serving, however, $\alpha = 2.0$ produces 16% less output than $\alpha = 0.1$.

FlashStates. FlashStates ties the best fixed value, $\alpha = 0.1$, which it exceeds by 2.9%, within two standard errors. It outperforms the value the tuner selects by 10.2% (pooled over two runs) and every $\alpha \geq 0.2$ by 7.4–18.8%. FlashStates also computes the fewest prompt tokens (2.06× the ideal versus 2.41–3.87×) and has the lowest p90 TTFT (1.05 s versus 1.84–3.53 s). Only $\alpha = 0$ has a lower median uncached prompt (1,025 versus 1,217 tokens). Marconi's throughput thus depends on a parameter that its own tuning objective barely distinguishes. FlashStates has no such parameter (Equation (7)), and its only fitted values, the prefill cost parameters, are set once per model (Appendix C).

F.2 Comparison with HiState

HiState (Pan et al., 2026) is concurrent work on recurrent checkpoints for agentic workloads. It observes that resuming from a cached prefix reads every KV entry of that prefix but only one recurrent checkpoint. HiState therefore prioritizes GPU and CPU memory for KV entries and keeps historical checkpoints on SSD. An adaptive interval controller sets how densely each new context segment is checkpointed. A unified eviction policy then ranks intermediate checkpoints together with the last KV segment of each trajectory and its final checkpoint.

Recurrent-checkpoint placement. ASC is orthogonal to SSD tiering approaches such as HiState. HiState adapts how densely it checkpoints within the block-based cache of vLLM, and its unlimited-cache reference counts reuse at the granularity of its cache blocks. ASC instead decouples recurrent-checkpoint placement from KV-cache block boundaries (Section 3), so a request can resume at the exact end of its predecessor. ASC therefore further reduces uncached tokens beyond what block-granular placement allows, even when an SSD tier retains every checkpoint. Such a tier could in turn hold the checkpoints that ASC materializes at any position.

Reuse model. HiState sets checkpoint intervals by assuming that a prefix hit is equally likely to end at any position inside a segment. FlashStates instead derives each request's ideal resume position from the prefixes processed before it (Equation (2)). In agent trajectories, a request usually extends the prompt and output of its predecessor, so this position is usually the exact end of the previous request. FlashStates then minimizes the prefill between actual and ideal resume positions under the cache budget (Equation (4)) and separates it into placement, capacity, and dependency misses (Equation (5)).

Eviction score. HiState adapts GreedyDual-Size by adding the last-access time of an entry to its expected recomputation per unit of reclaimed storage, with a time scale τ weighting the two terms. This parameter plays the role of Marconi's α. FlashStates uses the GreedyDual-Size watermark as its recency term, so recency and restore cost share units and need no weighting parameter (Equation (7)).

Evaluation setting. HiState evaluates Qwen3.5-9B on one H100 GPU with at most 24 live sessions and reports p95 and p99 TTFT together with token hit rates. Our evaluation serves three hybrid models with contexts of up to 1M tokens on two B200 GPUs per engine. It covers 24 to 48 concurrent coding-agent sessions (Section 5) and up to 80 with a CPU store (Appendix D), and it reports throughput together with TTFT.

G ASC Hybrid-State Lifecycle

Problem. Let $L = \texttt{num_computed_tokens}$ be a request's exact processed end and b_{KV} the KV-cache block size in tokens. If L lies inside a KV-cache block, request cleanup would release the live recurrent and auxiliary state, while the ordinary KV-cache-block-aligned path can name only an earlier resume position. Reusing L therefore requires complete hybrid state whose attention KV, recurrent checkpoint, and auxiliary state all identify the same exact prefix.

Mechanism. Before cleanup, ASC assembles this hybrid state from the request's existing cache allocations and makes it available to hybrid-state admission in Section 4.3. The capture retains the terminal KV-cache block,

marks only tokens before L as valid, and associates the matching recurrent checkpoint and auxiliary state with the same prefix identity and model configuration. Capture copies no model data. If admission retains the hybrid state, its components remain one logical recovery unit; otherwise, cache management may reuse allocations that no other hybrid state or live request needs.

How. For $L \bmod b_{\mathrm{KV}} \neq 0$, capture occurs before the request's KV-cache blocks return to the pool and succeeds only if every required cache group supplies the matching component. Aborted requests and request ends whose components are not fully materialized produce no hybrid state. For $L \bmod b_{\mathrm{KV}} = 0$, the special capture path for positions inside a KV-cache block does nothing; the ordinary KV-cache-block-aligned publication and lookup path remains responsible for any reusable state at L. This delegation does not guarantee that complete hybrid state exists: if any required component is absent, lookup safely falls back to an earlier position.

On reuse, the engine performs its normal prefix lookup and extends it to the longest matching admitted hybrid state whose components remain resident. Earlier KV-cache blocks may remain shared; the request receives private mutable recurrent and auxiliary state and a copy-on-write terminal KV-cache block. A missing or mismatched component leaves the normal lookup result unchanged.

Appendix H describes the kernel implementation, supported offsets, and per-call export limit of In-Chunk State Export.

H In-Chunk State Export: Kernel Implementation

Algorithm 1 describes one sequence with N input tokens in a prefill call. The tile $X[b{:}t]$ covers offsets b through $t-1$, and S is the recurrent state before that tile. The requested offset c is measured from the start of the call; if the call starts after q tokens, the absolute prefix position is $p = q + c$. In the pseudocode, the optional checkpoint output R is initially none. Without a valid supported offset, all export branches are skipped and R remains none.

OriginalTile performs the unmodified tile computation and returns its token outputs, next recurrent state S_{next}, and intermediate values Z. SerializeFinalState writes a recurrent state in the same checkpoint format and numerical representation used for the kernel backend's normal final-state output. At a tile boundary, the required recurrent state is already available and can be passed directly to SerializeFinalState.

KDA exports the state available at a 16-token tile boundary. GDN supports offsets aligned to 16 tokens within its 64-token tiles. For a GDN interior offset, PrefixState uses the state S before the tile and its original intermediates Z to construct the state after the first $c - b$ tokens of that tile. This requires an additional state update for that local prefix, rather than another prefill call over the full prefix. The result is written to R, while the original recurrence advances with S_{next}.

Algorithm 1 Fused One-Position State Export

$c \leftarrow$ validated offset supported by the kernel backend, otherwise none
for each kernel tile $X[b{:}t]$ in a call of length N **do**
  if $c = b$ **then** $R \leftarrow \textsc{SerializeFinalState}(S)$ **end if**
  $(O[b{:}t], S_{\mathrm{next}}, Z) \leftarrow \textsc{OriginalTile}(X[b{:}t], S)$
  if $b < c < t$ **then**
    $R \leftarrow \textsc{SerializeFinalState}(\textsc{PrefixState}(Z, S, c - b))$
  end if
  $S \leftarrow S_{\mathrm{next}}$
end for
if $c = N$ **then** $R \leftarrow \textsc{SerializeFinalState}(S)$ **end if**
return O, S, R

A checkpoint with d_S scalars adds one $4d_S$-byte fp32 output. KDA adds $\Theta(d_S)$ work to write the checkpoint in the kernel backend's output format, with no new matrix multiplication. A GDN interior offset adds one update for the prefix within that tile and $\Theta(d_S)$ on-chip movement of state data. Only one requested position is exported per sequence and prefill call. Calls without export retain the kernel backend's original computation.

In the main serving experiments (Section 5), in-chunk export is enabled only for the KDA kernel backend of GLM-5.3-Flash. The GDN exporter has kernel-level validation; Qwen3.8-Flash-Next and Nemotron 3 Super use exact request-completion capture without in-chunk export. The supplementary Nemotron experiment in Figure 11 uses the upstream Mamba2 checkpoint path for in-chunk export.

I Analysis of Hybrid State Management

This appendix states and proves the properties of an optimal policy that Section 4.2 lists. Appendix I.1 states the four assumptions and the model behind them, and Appendix I.2 proves the decomposition in Equation (5). Appendices I.3–I.6 state and prove Propositions 1 and 2, Lemma 1, and Proposition 3, which give the three properties. Appendix I.7 compares timers proportional to cost density with one shared timer (Proposition 4). Appendix I.8 relates GDS and LRU over whole hybrid states to these two timers, and Appendix I.9 lists what the analysis leaves out.

I.1 Model

A *conversation* is the sequence of requests that one agent issues, and a sub-agent forms its own conversation. The analysis rests on four simplifying assumptions.

1. **Exact continuation.** Each request of a conversation starts with the prompt and output of the conversation's previous request, followed by tokens that no request has processed after that prefix, such as a new tool result.
2. **Independent tool-call durations.** Tool-call durations are drawn independently from one distribution, regardless of context length or cache contents. We place each return at the lookup time of the next request. The *hazard rate* at time t is the return rate of a conversation that has waited for t.
3. **Separate conversations.** Conversations share no attention KV except a common prefix whose hybrid state stays cached.
4. **Fluid capacity.** We ignore the rounding of attention KV to whole KV-cache blocks and apply the budget to the occupied capacity averaged over time.

In our coding runs at 32 sessions, 96.9–98.1% of later requests start with the previous prompt and output, and the rest follow context rewrites recorded in the trajectories (Appendix C). Appendix E examines reuse under context changes in our Mooncake integration.

Conversation i issues requests $k = 1, 2, \ldots$, and request k has prompt length $n_{i,k}$. Let $\ell_{i,k}^{\text{end}}$ be the request-end position of request k, the length of the prefix that it has processed when it completes. Formally, under exact continuation, the prompt of request $k+1$ starts with this processed prefix and continues with tokens that no request has processed after this prefix. Formally, under separate conversations, the prompts of two conversations share at most a common prefix of length z, whose hybrid state stays cached. Then $z \le \ell_{i,k}^{\text{end}}$ for every request.

For the analysis, we write the prefill cost as $C(a,b) = \int_a^b c(x)\,dx$ with a nondecreasing per-token cost c. The cost model of Appendix C has this form with $c(x) = d + e\,x/10^5$.

The tool-call duration Γ has cumulative distribution Φ and survival function $\bar{\Phi} = 1 - \Phi$. When Φ has a density ϕ, the hazard rate is $\zeta(t) = \phi(t)/\bar{\Phi}(t)$. A nondecreasing hazard rate describes durations that become more likely to end as they grow (e.g., near-deterministic tools), and a decreasing one describes durations in which a long wait predicts a longer one. An exponential duration has a constant hazard rate.

Capacity accounting. $\mathcal{M}_\pi(\tau)$ counts each distinct cache allocation once, including the private mutable copies that live requests own. A shared KV-cache block counts once even when several hybrid states require it. The terminal KV-cache block of a hybrid state (Section 3.1) counts at its full size, although only its tokens before the endpoint are valid. Eviction frees an allocation only when no intact cached hybrid state or live request still requires it. A newly materialized hybrid state takes its allocations from the request that materializes it (Section 3). A request reuses allocations only through the hybrid state from which it resumes. With GPU memory alone, the budget applies independently to each GPU. With a host-memory tier, the formulation

treats both tiers as one pool, and C counts only prefill, not the time to load allocations back into GPU memory.

I.2 Decomposition of Redundant Prefill

Equation (5) follows from C's additivity once $\ell_r^\pi \le \ell_r^{\mathrm{kv}} \le \ell_r^{\mathrm{mat}} \le \ell_r^\star$. The last two inequalities hold by definition. If $\ell_r^\pi = 0$, the first one holds trivially. Otherwise, $\ell_r^\pi = p(u)$ for a usable hybrid state u. The prefix of u was processed before τ_r and matches X_r, so $p(u) \le \ell_r^\star$, and u was materialized at $p(u)$. Its allocations, including its attention KV, are resident at τ_r. Position $p(u)$ therefore satisfies the definition of ℓ_r^{kv}, and $\ell_r^\pi \le \ell_r^{\mathrm{kv}}$.

I.3 Materializing at Request Ends

Proposition 1. *Under exact continuation and separate conversations, the ideal resume position of each later request equals the request-end position of the previous request of its conversation, namely the number of tokens that the previous request processed. Materializing hybrid state at every request end therefore makes the placement term zero. Suppose that hybrid state can be materialized only at KV-cache block boundaries, which lie every β tokens. The placement term of request r is then at least $C(\beta\lfloor \ell_r^\star/\beta \rfloor, \ell_r^\star)$, even when every boundary is materialized.*

Proof. Let request r be request $k+1$ of conversation i, and let Z be the prefix that request k processed, of length $\ell_{i,k}^{\mathrm{end}}$. Z belongs to $\mathcal{P}_r$, and X_r starts with Z, so $\ell_r^\star \ge \ell_{i,k}^{\mathrm{end}}$. A processed prefix that matches X_r beyond $\ell_{i,k}^{\mathrm{end}}$ would contain Z followed by the first of the new tokens. Conversation i processed no such prefix before τ_r, because its earlier requests processed only prefixes of Z. Another conversation shares at most the common prefix of length $z \le \ell_{i,k}^{\mathrm{end}}$ with X_r. Hence $\ell_r^\star = \ell_{i,k}^{\mathrm{end}}$.

If the engine materializes hybrid state at every request end, then $\ell_r^{\mathrm{mat}} = \ell_{i,k}^{\mathrm{end}} = \ell_r^\star$, and the placement term $C(\ell_r^{\mathrm{mat}}, \ell_r^\star)$ is zero. If it can materialize hybrid state only at KV-cache block boundaries, every β tokens, then ℓ_r^{mat} is a multiple of β that does not exceed $\ell_r^\star$, so $\ell_r^{\mathrm{mat}} \le \beta\lfloor \ell_r^\star/\beta \rfloor$. Because C is additive and nonnegative, $C(\ell_r^{\mathrm{mat}}, \ell_r^\star) \ge C(\beta\lfloor \ell_r^\star/\beta \rfloor, \ell_r^\star)$. vLLM materializes hybrid state only at multiples of β inside prompts, and the longest prompt that conversation i processed before τ_r is that of request k, with length $n_{i,k}$. Its placement term is therefore at least $C(\beta\lfloor n_{i,k}/\beta \rfloor, \ell_{i,k}^{\mathrm{end}})$, which covers every output token of request k that was processed. □

I.4 Eviction Without Dead Allocations

As in Section 4.1, a cached hybrid state is intact when all of its allocations are resident, and only an intact hybrid state can be usable. A resident allocation is dead when no live request and no intact cached hybrid state requires it.

Proposition 2. *(i) For every policy in Π_{ASC}, some policy in Π_{ASC} gives every request the same actual resume position, never occupies more capacity, holds no dead allocation, and removes hybrid states only by evicting whole sets $F(u)$. (ii) For a cached hybrid state u with $s(u) > 0$, $F(u)$ is the smallest set of allocations whose eviction makes u unusable without leaving a dead allocation. (iii) Under exact continuation and separate conversations, a policy that holds no dead allocation has a zero dependency term.*

Proof. **Part (i).** Take any policy $\pi \in \Pi_{\mathrm{ASC}}$. Define π' to hold, at every time τ, exactly the allocations required by the live requests and by the hybrid states that are cached and intact under π at τ. π holds all of these allocations, so π' never occupies more capacity than π. The two policies have the same intact hybrid states at every time. A newly materialized hybrid state takes its allocations from the request that materializes it, and that request reuses allocations only through the hybrid state from which it resumes, so the live requests are also the same under both policies. Every request r therefore has the same usable set, $\mathcal{U}_r^\pi = \mathcal{U}_r^{\pi'}$, and the same actual resume position. π' holds no dead allocation by construction. Besides following the live requests and the newly materialized hybrid states, π' releases allocations when π removes a hybrid state u from the intact set, and it then releases the allocations that no other intact hybrid state or live request requires, which is $F(u)$. When π removes several hybrid states at once, π' releases their sets $F(\cdot)$ one after another, each

computed after the previous release. π' decides from the cache contents under π alone, so it is causal, and it retains hybrid state only where π does, so $\pi' \in \Pi_{\mathrm{ASC}}$.

Part (ii). Let an eviction release a set E of allocations that contains an allocation of $D(u)$, so that u becomes unusable, and suppose that it leaves no dead allocation. After the eviction, no intact hybrid state and no live request requires an allocation of $F(u)$, because before it only u did. Every allocation of $F(u)$ must therefore be in E, so $E \supseteq F(u)$. Conversely, $E = F(u)$ contains an allocation of $D(u)$ when $s(u) > 0$. It contains no allocation of another hybrid state or live request, so every other intact hybrid state stays intact, and every allocation outside $F(u)$ keeps the requirement that it had. $F(u)$ is therefore the smallest such set.

Part (iii). Suppose that some request r has $\ell_r^\pi < \ell_r^{\mathrm{kv}} = p$ under a policy that holds no dead allocation. If $p \le z$, the hybrid state of the common prefix is cached and usable for r, so $\ell_r^\pi \ge z \ge p$, a contradiction. Otherwise, the attention KV of the token before position p is resident, and since it is not dead, an intact hybrid state w or a live request requires it. Under separate conversations, only requests and hybrid states of r's conversation hold attention KV of X_r beyond the common prefix. At τ_r, no other request of that conversation is live, and r holds no allocation before its lookup. Hence w is an intact hybrid state of r's conversation whose prefix contains $X_r[0{:}p)$, so $p(w) \ge p$. Under exact continuation, every prefix that the conversation has processed is a prefix of X_r, so w is usable for r, and $\ell_r^\pi \ge p(w) \ge p$, a contradiction. The dependency term $C(\ell_r^\pi, \ell_r^{\mathrm{kv}})$ is therefore zero for every request. □

I.5 Cost of an Eviction

Lemma 1. *Let a policy evict cached hybrid state u by releasing exactly $F(u)$, with all other decisions fixed. The eviction changes the actual resume position only of the requests that would resume from u. Each of them resumes at $f(u)$ instead, so its redundant prefill grows by exactly $g(u)$, whatever its ideal resume position. Here, $f(u)$ and $g(u)$ are taken at that request's lookup time. The eviction releases exactly $s(u)$ KV-cache blocks.*

Proof. Compare the run that keeps u with the run that evicts u by releasing $F(u)$, all other decisions being the same. $F(u)$ contains no allocation that another intact cached hybrid state or live request requires, so every other hybrid state is usable in one run exactly when it is usable in the other. For each later request r, the usable set in the evicting run is therefore the usable set in the keeping run without u. If r would not resume from u in the keeping run, the maximum in Equation (3) does not change. If it would, the remaining usable position nearest to the ideal resume position lies before $p(u)$ on the prefix of u, so it is the fallback position $f(u)$ at τ_r. The redundant prefill of r changes from $C(p(u), \ell_r^\star)$ to $C(f(u), \ell_r^\star)$. Additivity gives $C(f(u), \ell_r^\star) - C(p(u), \ell_r^\star) = C(f(u), p(u)) = g(u)$, which does not depend on $\ell_r^\star$. The eviction releases the $|F(u)| = s(u)$ KV-cache blocks of $F(u)$. □

I.6 Optimal Timers

Timer model. We first state the timer model of Section 4.2 precisely. Each conversation that waits for a tool has at most one cached hybrid state, its latest, and it eventually returns. Under separate conversations, the fallback position of this hybrid state is z, the length of the common prefix. Its restore cost g and released capacity $s > 0$ therefore stay fixed while it is cached (Lemma 1). Hybrid states arrive, one at each request completion, at a long-run rate λ. Their pairs (g, s) follow one distribution with $\mathbb{E}[g] < \infty$, independent of tool-call durations. We also assume that the time to serve a request is negligible, so this arrival process does not depend on the policy. A policy sets a timer $h \in [0, \infty]$ for each hybrid state when its request completes. The timer is a fixed, possibly randomized, function of (g, s) and does not depend on Γ. The hybrid state stays cached for $\min(\Gamma, h)$, whose mean is $W(h) = \int_0^h \bar{\Phi}(t)\, dt$. It saves g exactly when $\Gamma \le h$, an event of probability $\Phi(h)$. We assume $\Phi(0) = 0$ and $\mathbb{E}[\Gamma] < \infty$. By Little's law, the time-averaged occupied capacity is the arrival rate times the mean product of capacity and cached time per arrival, $\lambda\, \mathbb{E}[s\, W(h)]$. The prefill saved per unit time is $\lambda\, \mathbb{E}[g\, \Phi(h)]$. Under fluid capacity, the policy therefore solves

$$\max_h \ \mathbb{E}\big[g\, \Phi(h)\big] \quad \text{subject to} \quad \mathbb{E}\big[s\, W(h)\big] \le \kappa/\lambda, \tag{8}$$

where κ is the number of KV-cache blocks of the budget that remain for these hybrid states after live requests and the common prefix.

Proposition 3. *In the timer model, some optimal policy sets the timer of each hybrid state from its cost density alone, and the timer is nondecreasing in the cost density. To fill the budget exactly, the policy may give hybrid states of one cost density either of two timers at random. If the tool-call duration is deterministic or has a nondecreasing hazard rate, this policy is a threshold rule. Above a cost-density threshold set by the budget, it keeps a hybrid state until its conversation returns. Below the threshold, it gives the hybrid state a zero timer. If the hazard rate decreases, the timer of a hybrid state expires when the hazard rate falls to a level inversely proportional to its cost density.*

Proof. **Maximizers.** For a price $\mu \geq 0$ per KV-cache block per unit time of occupancy, let

$$\psi(h; \rho) = \rho\, \Phi(h) - \mu\, W(h). \tag{9}$$

For a hybrid state with cost density $\rho = g/s$, $s\,\psi(h;\rho)$ is its expected saving minus the price of its expected occupancy. The function $\psi(\cdot;\rho)$ is upper semicontinuous on $[0,\infty]$, because Φ is right-continuous and nondecreasing and W is continuous with $W(\infty) = \mathbb{E}[\Gamma]$. Its maximizers therefore form a nonempty closed set, and limits of maximizers are maximizers. They depend on a hybrid state only through its cost density.

Lagrangian relaxation. Let h^μ give every hybrid state a maximizer of $\psi(\cdot;\rho)$, possibly chosen at random. Suppose that h^μ meets the constraint of Equation (8), with equality when $\mu > 0$. Then h^μ is optimal. For any feasible h, the constraint gives $\mathbb{E}[g\,\Phi(h)] \leq \mathbb{E}[s\,\psi(h;\rho)] + \mu\kappa/\lambda$. The right side is at most $\mathbb{E}[s\,\psi(h^\mu;\rho)] + \mu\kappa/\lambda$. This bound equals $\mathbb{E}[g\,\Phi(h^\mu)]$, because h^μ meets the constraint with equality or $\mu = 0$.

Such a policy exists. Every maximizer has $\psi(h;\rho) \geq \psi(0;\rho) = 0$, so $\mu\, s\, W(h) \leq g$. The mean occupancy $\mathbb{E}[s\,W(h)]$ is thus at most $\mathbb{E}[g]/\mu$, which tends to 0 for large μ. Exchanging maximizers for two prices shows that the mean occupancy does not increase with μ. Because limits of maximizers are maximizers, consider the price at which the mean occupancy crosses κ/λ. There, the smallest maximizers occupy at most κ/λ and the largest at least κ/λ. Mixing them at random meets the constraint with equality, as in the fractional knapsack problem. If keeping every hybrid state until its conversation returns fits the budget, $\mu = 0$ works. An optimal policy therefore gives every hybrid state a maximizer at such a price, mixing maximizers only so that the constraint holds with equality when $\mu > 0$.

Dependence on cost density only. Let $\rho_2 > \rho_1$, and let h_1 and h_2 maximize $\psi(\cdot;\rho_1)$ and $\psi(\cdot;\rho_2)$ with $h_2 < h_1$. Adding $\psi(h_1;\rho_1) \geq \psi(h_2;\rho_1)$ and $\psi(h_2;\rho_2) \geq \psi(h_1;\rho_2)$ gives $(\rho_2 - \rho_1)(\Phi(h_2) - \Phi(h_1)) \geq 0$, so $\Phi(h_2) = \Phi(h_1)$. Then $\psi(h_2;\rho_1) - \psi(h_1;\rho_1) = \mu \int_{h_2}^{h_1} \bar{\Phi}(t)\, dt \geq 0$. Since h_1 is a maximizer, this difference is 0. If $\mu > 0$, $\bar{\Phi}$ vanishes on (h_2, h_1) and hence, being right-continuous, at h_2. If $\mu = 0$, every maximizer for $\rho > 0$ has $\Phi(h) = 1$. In both cases, both timers outlast every tool call and act identically. Any choice of maximizers, including a randomized one, thus gives a timer that is nondecreasing in ρ, apart from timers that outlast every tool call.

Deterministic duration or nondecreasing hazard rate. Let $\hat{t}$ be the supremum of the support of Γ, possibly infinite. On $[0, \hat{t})$, $\psi'(h) = \bar{\Phi}(h)\,(\rho\,\zeta(h) - \mu)$, and $\rho\,\zeta(h) - \mu$ is nondecreasing. The function ψ therefore first decreases and then increases. Its maximum lies at $h = 0$, where $\psi = 0$, or at $h = \hat{t}$, where $\psi = \rho - \mu\,\mathbb{E}[\Gamma]$. A timer beyond $\hat{t}$ changes nothing, because the conversation has returned by then. The optimal policy therefore keeps a hybrid state until reuse if $\rho > \mu\,\mathbb{E}[\Gamma]$ and gives it a zero timer if $\rho < \mu\,\mathbb{E}[\Gamma]$. At equality, it may mix the two to use the budget fully. A deterministic duration $\Gamma \equiv \gamma_0 > 0$ has no density. However, $\psi(h) = -\mu h$ for $h < \gamma_0$ and $\psi(h) = \rho - \mu\gamma_0$ for $h \geq \gamma_0$ give the same rule with $\mathbb{E}[\Gamma] = \gamma_0$.

Decreasing hazard rate. Now $\rho\,\zeta(h) - \mu$ is nonincreasing, so ψ first increases and then decreases. The optimal timer is $h^\star(\rho) = \inf\{h \geq 0 : \zeta(h) \leq \mu/\rho\}$, the time at which the hazard rate falls to μ/ρ. It is nondecreasing in ρ. It is finite when $\rho \lim_{t\to\infty} \zeta(t) < \mu$, in particular for every ρ when the hazard rate tends to zero. For example, a Lomax (Pareto type II) duration with $\bar{\Phi}(t) = (1 + t/t_0)^{-\eta}$ and $\eta > 1$ has $\zeta(t) = \eta/(t_0 + t)$, so $h^\star(\rho) = \max\{0,\ \eta\rho/\mu - t_0\}$. □

This proof follows the hazard-rate analysis of TTL caches by Ferragut et al. (2016), here with a restore cost and a released capacity for every hybrid state. Suppose instead that a conversation ends after a request with probability ω, independent of everything else. Then Φ becomes $(1-\omega)\Phi$, and $W(\infty)$ becomes infinite. For $\mu > 0$, the maximizers are finite, and the exchange argument still shows that the optimal timer is nondecreasing in ρ. The threshold rule no longer holds, because keeping a hybrid state until its conversation returns would keep it forever with probability ω.

I.7 Proportional Versus Shared Timers

Recall the hazard rate ζ of Appendix I.1. We call $t\,\zeta(t)$ the *generalized failure rate*, as failure rate is another name for hazard rate. A nondecreasing $t\,\zeta(t)$ is known as an *increasing generalized failure rate* (Lariviere, 2006). Informally, a conversation that has waited longer is then at least as likely to return within the next 1% of its wait so far.

Proposition 4. *In the timer model, let tool-call durations have an increasing generalized failure rate and a probability density that is positive on $(0, \infty)$. Consider two policies that keep the same number of KV-cache blocks occupied on average. The first gives every hybrid state the same timer. The second gives each hybrid state a timer proportional to its cost density. The second policy saves at least as much prefill as the first, and strictly more when cost densities differ.*

Proof. We use the timer model and Equation (8). The first policy sets the timer h_0 for every hybrid state, and the second sets $h = \rho/\theta$ for a constant $\theta > 0$. Both occupy the same capacity, $\mathbb{E}[s\,W(\rho/\theta)] = \mathbb{E}[s]\,W(h_0)$. Their saving rates differ by $\lambda\,\mathbb{E}[g\,(\Phi(\rho/\theta) - \Phi(h_0))]$. Let $\rho_0 = \theta h_0$, the cost density whose proportional timer equals h_0.

Consider a hybrid state with $\rho > \rho_0$, so that $\rho/\theta > h_0$. Because $\phi = \zeta\,\bar{\Phi}$,

$$\rho\,\big(\Phi(\rho/\theta) - \Phi(h_0)\big) = \int_{h_0}^{\rho/\theta} \rho\,\zeta(t)\,\bar{\Phi}(t)\,dt.$$

For t in this range, $\rho \geq \theta t$, and $t\,\zeta(t) \geq h_0\,\zeta(h_0)$ because $t\,\zeta(t)$ is nondecreasing. Hence $\rho\,\zeta(t) \geq \theta\,t\,\zeta(t) \geq \rho_0\,\zeta(h_0)$, and

$$\rho\,\big(\Phi(\rho/\theta) - \Phi(h_0)\big) \;\geq\; \rho_0\,\zeta(h_0)\,\big(W(\rho/\theta) - W(h_0)\big). \tag{10}$$

Now consider a hybrid state with $\rho < \rho_0$. For t in $[\rho/\theta, h_0]$, the same steps give $\rho\,\zeta(t) \leq \theta\,t\,\zeta(t) \leq \rho_0\,\zeta(h_0)$. Integrating over this interval gives $\rho\,(\Phi(h_0) - \Phi(\rho/\theta)) \leq \rho_0\,\zeta(h_0)\,(W(h_0) - W(\rho/\theta))$, which is Equation (10) multiplied by -1. For $\rho = \rho_0$, both sides of Equation (10) are 0. Multiplying Equation (10) by s and taking expectations gives

$$\mathbb{E}\big[g\,(\Phi(\rho/\theta) - \Phi(h_0))\big] \;\geq\; \rho_0\,\zeta(h_0)\,\big(\mathbb{E}[s\,W(\rho/\theta)] - \mathbb{E}[s]\,W(h_0)\big) = 0,$$

so the second policy saves at least as much prefill.

If cost densities differ, hybrid states with $\rho > \rho_0$ occur with positive probability. Otherwise, $\rho/\theta \leq h_0$ almost surely and $\rho/\theta < h_0$ with positive probability. The second policy would then occupy less capacity, because W increases strictly when Γ has a positive density. For a hybrid state with $\rho > \rho_0$, $\rho > \theta t$ and $\zeta(t) > 0$ for $t < \rho/\theta$. The first inequality above is therefore strict on $(h_0, \rho/\theta)$, and so is Equation (10). The second policy then saves strictly more. This proof does not use $\mathbb{E}[\Gamma] < \infty$. □

Distributions with an increasing generalized failure rate. The generalized failure rate $t\,\zeta(t)$ is the derivative of the cumulative hazard $-\ln\bar{\Phi}(t)$ with respect to $\ln t$. It is nondecreasing exactly when the cumulative hazard is convex in $\ln t$. Then, for every fraction $\delta > 0$, the probability $1 - \bar{\Phi}((1+\delta)t)/\bar{\Phi}(t)$ of returning within the next fraction δ of the time already waited does not decrease with t. The following durations, each with scale t_0, satisfy the condition. An exponential duration has $t\,\zeta(t) = t/t_0$. A Weibull duration with $\bar{\Phi}(t) = \exp(-(t/t_0)^\nu)$ has $t\,\zeta(t) = \nu\,(t/t_0)^\nu$. A log-logistic duration with $\bar{\Phi}(t) = 1/(1+w)$ and $w = (t/t_0)^\nu$ has $t\,\zeta(t) = \nu\,w/(1+w)$. A Lomax duration has $t\,\zeta(t) = \eta\,t/(t_0 + t)$. A lognormal duration whose logarithm has mean $\ln t_0$ and standard deviation σ has $t\,\zeta(t) = \zeta_{\mathcal{N}}(\ln(t/t_0)/\sigma)/\sigma$. Here, the hazard rate $\zeta_{\mathcal{N}}$ of the standard normal distribution is increasing. A gamma duration with shape ν has $t\,\zeta(t) = v^\nu \exp(-v)/U_\nu(v)$ with $v = t/t_0$ and $U_\nu(v) = \int_v^\infty y^{\nu-1}\exp(-y)\,dy$. For $\nu \geq 1$, its hazard rate is nondecreasing, so $t\,\zeta(t)$ increases. For $\nu < 1$, $y^{\nu-1} \leq v^{\nu-1}$ for $y \geq v$ gives $U_\nu(v) \leq v^{\nu-1}\exp(-v)$. The derivative $\nu/v - 1 + v^{\nu-1}\exp(-v)/U_\nu(v)$ of $\ln(t\,\zeta(t))$ with respect to v is then at least $\nu/v > 0$. In every case, $t\,\zeta(t)$ increases in t.

Conversations that end. With the ending probability ω of Appendix I.6, the tool-call duration is infinite with probability ω. Then $t\,\zeta(t)$ cannot stay nondecreasing, because otherwise $\bar{\Phi}$ would fall to 0, below ω. The proof uses the condition only for t between the smallest and the largest of h_0 and the timers ρ/θ. The result therefore still holds when $t\,\zeta(t)$ is nondecreasing on that interval.

I.8 GDS and LRU in the Timer Model

GDS. A hybrid state that no request reuses is evicted when the watermark reaches its priority. By then, the watermark has risen by ρ since the stamp of the hybrid state (Equation (7)). If the watermark rises at a steady rate $\dot{\Lambda} > 0$ while the hybrid state is cached, GDS therefore sets the timer $\rho/\dot{\Lambda}$. This is the second policy of Proposition 4 with $\theta = \dot{\Lambda}$. We take this steady rate as an approximation for a large cache with many conversations. Like the optimal timer of Proposition 3, this timer depends only on the cost density and increases with it. GDS sets it without knowing Φ or the price μ. Admission keeps every newly materialized hybrid state except one that would be the next to leave. Under a steady watermark, this exception applies only to cost densities close to zero, so admission changes the timer only for such hybrid states.

Whole-state LRU. LRU over whole hybrid states, which we call *whole-state LRU*, evicts them in the order of their last use, regardless of their cost density. In a large cache, every hybrid state therefore stays cached for about the same time after its last use, the characteristic time of the cache (Che et al., 2002; Fricker et al., 2012). This is the first policy of Proposition 4. Suppose that the budget is full under both policies and that the two approximations hold. Proposition 4 then implies that GDS saves at least as much prefill as whole-state LRU for durations with an increasing generalized failure rate. It saves strictly more when cost densities differ.

I.9 Scope of the Analysis

The analysis leaves out several effects that the online policy still handles. First, conversations can share attention KV beyond a common prefix, for example when a sub-agent or a forked conversation starts from another conversation's prefix. $s(u)$ then excludes shared allocations and $f(u)$ depends on other conversations, which FlashStates tracks (Appendix J) but the model does not. Second, tool-call durations differ across tools and conversations, so a conversation's own duration distribution also matters; GDS observes it only through the stamps of reused hybrid states. Third, KV-cache blocks and hybrid states are discrete, so Equation (8) becomes a knapsack problem, and the fluid results hold approximately when each hybrid state is small relative to the budget. Fourth, a hit shortens a request's prefill and thus changes when its conversation issues the next request, which the analysis ignores. Finally, agents that edit earlier context violate exact continuation; Appendix E evaluates retaining recent checkpoints for these requests.

J Admission and Eviction Details

This appendix details the admission and eviction of Section 4.3.

Admission. A *free* KV-cache block holds no resident allocation. If vLLM's next KV-cache block to allocate is free, keeping a newly materialized hybrid state displaces nothing, and FlashStates retains it directly. Otherwise, FlashStates compares its priority (Equation (7)) with the minimum priority among the eligible cached hybrid states. If its priority is lower, GDS would evict it next, so FlashStates does not admit it. Otherwise, it remains cached. A later request with an exactly matching prefix can then reuse it while it stays intact. In the timer model of Appendix I.6, a hybrid state that FlashStates does not admit has a zero timer.

Eviction. When a request needs new KV-cache blocks, FlashStates first takes free blocks and then evicts eligible hybrid states in the order of their priority. A cached hybrid state can stop being intact, for example when no eligible hybrid state is left and FlashStates reclaims blocks in vLLM's least-recently-used order. Neither a dead allocation nor such a hybrid state can serve a request. FlashStates gives a dead allocation the current watermark as its priority, and a hybrid state that is no longer intact has restore cost 0, so its priority equals its stamp. Both priorities are at most the watermark, so these allocations leave before any intact hybrid state whose priority exceeds the watermark.

Priority updates. Only a use refreshes the stamp of a hybrid state. Between uses, the hybrid state ages as Λ rises. Evicting a hybrid state can change the fallback position of hybrid states that extend its prefix, and changes in KV-cache-block sharing can change released capacity. FlashStates then updates $g(u)$, $s(u)$, and $H(u)$ of the affected hybrid states but not $\text{stamp}(u)$, because such a change is not a use of u.